

CP References — ICPC Team Notebook

Contest Notes	2
Setup & don't-forget	2
Number theory	2
Bits	2
Games (Grundy)	2
Expected value	2
Misc	2
Tricks	3
Number Theory	4
divisors.cpp	5
euler-phi.cpp	6
ext-gcd-crt.cpp	7
linear-sieve.cpp	8
miller-rabin.cpp	9
sieve.cpp	10
Combinatorics	11
combinatorics.cpp	12
euler-pentagonal-partitions.cpp	13
ncr-pascal.cpp	14
DP	15
digit-dp.cpp	16
lis.cpp	17
matrix-exponentiation.cpp	18
rerooting.cpp	19
sos-dp.cpp	20
tree-knapsack.cpp	21
Graphs & Trees	22
articulation-points.cpp	23
bellman-ford.cpp	24
bridges-using-tarjan.cpp	25
bridge-tree.cpp	26
dijkstra.cpp	27
dsu.cpp	28
dsu-rollback.cpp	29
floyd-warshall.cpp	30
hld.cpp	31
lca.cpp	33
mst-kruskal.cpp	34
scc.cpp	35
topological-sort.cpp	36
Range Queries	37
fenwick-tree-updatepoint-queryrange.cpp	38
fenwick-tree-updaterange-querypoint.cpp	39
lazy-seg-tree.cpp	40
mo.cpp	42
mo-on-tree.cpp	43
mo-with-rollback.cpp	45
seg-tree.cpp	47
seg-tree-iterative.cpp	48
sliding-window-minmax.cpp	49
sparse-table.cpp	50
sqrt-decomposition.cpp	51
Strings	52
binary-trie.cpp	53
hashing.cpp	54
kmp.cpp	55
manacher.cpp	56
multiset-hashing.cpp	57
trie.cpp	58
Geometry	59
closest-pair.cpp	60
geometry-basics.cpp	61
Other	62
bit-tricks.cpp	63
coordinate-compression.cpp	64
ordered-multiset.cpp	65
random.cpp	66

Contest Notes

Quick facts that solve or unlock problems. (Code lives in the folders; see README index.)

Setup & don't-forget

- **Fast IO** — first lines of main: `ios_base::sync_with_stdio(false); cin.tie(nullptr);` — after this NEVER mix in `scanf/printf`.
- `'\n'` not `endl` (`endl` flushes = slow in loops). **Exception: interactive problems** — there you MUST flush every query (`endl` or `cout.flush()`).
- **Interactive protocol:** `cout << "? " << x << endl;` then read the reply — `endl` everywhere, no buffering games. Note `cin.tie(nullptr)` disables the auto-flush-before-read, so with fast IO every un-flushed query = guaranteed idleness TLE.
- If the judge replies `-1` (or anything out of protocol) $\rightarrow$ `exit(0)` **immediately:** you exceeded the query budget or sent garbage, and looping on turns a clean WA into a confusing TLE.
- Finish with `! answer + flush;` count queries against the budget first (it's usually $\sim \log_2 n$ — an off-by-one loop bound is the classic interactive WA).
- Multi-test skeleton: `int t; cin >> t; while (t--) solve();` — **reset globals/arrays inside solve()**, half of all WAs on test 2 are stale state.
- Real-number answers: `cout << fixed << setprecision(10);` — default prints 6 significant digits and truncates big values into scientific notation.
- Overflow: `1e5 * 1e5` overflows `int` **silently** — cast first: `(1ll)a * b`. Shifts: `1LL << k`, not `1 << k`, for $k \geq 31$. Constants: `const ll INF = 4e18;` (`int` max $\sim 2.1e9$).
- Big arrays **global**, never inside main — local ones overflow the stack. Same for recursion deeper than $\sim 1e5$ on some judges.
- `v.size()` is unsigned: `v.size() - 1` on an empty vector = huge number — write `(int)v.size()`.
- `sort` comparator must be **strict** (`<`, never `<=`) — a non-strict one is undefined behavior (can RE, not just mis-sort).
- Read until EOF: `while (cin >> x) { ... }`. File IO when the judge wants it: `freopen("in.txt", "r", stdin); freopen("out.txt", "w", stdout);`
- Compile: `g++ -O2 -std=c++17 a.cpp -o a` — while debugging add `-g -fsanitize=address,undefined` (catches overflow, out-of-bounds, UB at runtime).

Number theory

- The number of divisors of N is **odd iff N is a perfect square** (divisors pair up except $\sqrt{N}$). $\rightarrow$ `Number Theory/divisors.cpp`
- All common divisors of a, b are exactly the divisors of **`gcd(a, b)`**.
- Starting from 0 and repeatedly adding `STEP mod MOD`, you can reach **every** value $0..MOD-1$ **iff `gcd(STEP, MOD) = 1`**.
- Every even number > 2 is a sum of two primes (Goldbach — a conjecture, but verified far past any contest limit).
- The largest gap between consecutive primes below $1e9$ is only **~ 300** $\rightarrow$ "find a prime near x " loops terminate almost instantly.
- N is divisible by 6 iff N is even **and** its digit sum is divisible by 3.
- Euler's theorem: **$a^{\phi(m)} \equiv 1 \pmod{m}$** when $\gcd(a, m) = 1$.
- Huge exponents: **$a^n \equiv a^{(n \bmod \phi(m) + \phi(m))} \pmod{m}$** for $n \geq \log_2(m)$ — safe even when $\gcd(a, m) \neq 1$; drop the $+\phi(m)$ only if $\gcd = 1$. $\rightarrow$ `Number Theory/euler-phi.cpp`

Bits

- `n & (n-1)` — turn off the rightmost set bit (also: loop it to count set bits / test power of two).
- `n & (n+1)` — clear all trailing ones.
- `n | (n+1)` — set the rightmost zero bit.
- `n & -n` — extract the lowest set bit; note it always **divides n** .
- Number of set bits in $[1, 2^k - 1] = k \cdot 2^{k-1}$.
- Count of numbers in $[l, r]$ with a given bit set: $O(1)$ per bit. $\rightarrow$ `Other/bit-tricks.cpp`
- Prefix XOR $1^2^3 \dots^n$ is periodic mod 4: **$n, 1, n+1, 0$** for $n \% 4 = 0, 1, 2, 3$. $\rightarrow$ `Other/bit-tricks.cpp`
- N is a sum of K powers of two (repeats allowed) **iff `popcount(N)  $\leq K \leq N$`** .

Games (Grundy)

- Grundy number of a position = **mex of the Grundy numbers of positions reachable in one move**. Position is LOSING for the player to move **iff $g = 0$** . Compute by memoized DFS over the game graph.
- **Sum of independent games** (each turn: move in exactly ONE component): total $g =$ **XOR of the components' g** — losing iff the XOR is 0. This is why single-game Grundy tables solve multi-pile problems.
- **Nim** (take any amount from one pile): $g(\text{pile of } n) = n \rightarrow$ first player wins iff XOR of piles $\neq 0$. Winning move: pick a pile where $p \wedge \text{total} < p$, shrink it to $p \wedge \text{total}$.
- Subtraction game (take $1..k$): **$g(n) = n \bmod (k+1)$** . Unknown small game? **Brute-force $g(n)$ for $n \leq 60$ and look for the period** — most simple games are eventually periodic.
- **Misère nim** (who takes last LOSES): play exactly like normal nim, EXCEPT when every pile has size 1 — then first wins iff the number of piles is **even**.
- **Staircase nim** (move stones one step down a staircase): only stones on **odd steps** matter — XOR those pile sizes.
- No Grundy needed if you can find a **mirroring/pairing strategy**: copy the opponent's move in the symmetric half — proves a win without any computation. Also try "strategy stealing" for existence proofs.

Expected value

- **Linearity of expectation:** $E[X+Y] = E[X] + E[Y]$ — **works even when dependent**. The main move: write the target as a sum of 0/1 indicators, then $E[\text{total}] = \sum P(\text{indicator} = 1)$. ("Expected # of visible towers / inversions / adjacent equal pairs...")
- Nonnegative integer X : **$E[X] = \sum P(X \geq k)$, $k = 1, 2, \dots$** — often much easier than finding the full distribution ("expected number of rounds survived").
- Waiting times: success probability $p \rightarrow$ expected tries = **$1/p$** . **Coupon collector:** expected draws to see all n types = $n \cdot (1 + 1/2 + \dots + 1/n) \approx n \ln n$.
- k independent uniform $[0, 1]$: **$E[\min] = 1/(k+1)$, $E[\max] = k/(k+1)$** , i -th smallest = $i/(k+1)$. Same fractions for " k random cuts of a stick".
- Random permutation of n : expected **fixed points** = **1** (any $n!$), expected **# cycles** = **$1 + 1/2 + \dots + 1/n$** , $P(\text{two given elements share a cycle}) = 1/2$.
- Random process with states $\rightarrow$ set unknowns $E[\text{state}] =$ expected steps to finish, write one linear equation per state, solve: telescoping for chains/lines, **Gaussian elimination** for $\leq$ a few hundred states.
- **Gambler's ruin** (fair ± 1 walk from i , absorbing at 0 and n): $P(\text{reach } n \text{ first}) = i/n$; expected steps = **$i \cdot (n-i)$** .

Misc

- Chessboard $N \times M$: number of white squares = **$(N \cdot M + fg) / 2$** where $fg = 1$ if the corner square is white, else 0.

- A binary string with no palindromic substring of length > 1 has length ≤ 2 (adjacent chars must differ AND $s[i] \neq s[i+2]$ — contradiction from length 3 on).
- Any swap in a permutation changes the inversion count by an **odd** amount (flips parity). So 3-cycles preserve parity, and a duplicate value works as a "free parity reset" (swapping equal elements changes nothing).
- Minimum swaps to sort a permutation = $n - (\text{number of cycles in it})$.

Tricks

"See X $\rightarrow$ reach for Y" recipes. Skim when a problem feels unapproachable.

Picking the attack

- "**Minimize the maximum**" / "maximize the minimum" / "k-th smallest value" $\rightarrow$ **binary search on the answer**: write `possible(x)`, it's monotonic; for k-th smallest, count how many $\leq$ mid.
- "Count subarrays with sum/xor = k" $\rightarrow$ **prefix + hashmap**: $\text{pre}[r] - \text{pre}[l-1] = k$ means look up $\text{pre}[r] - k$ among earlier prefixes (xor: $\text{pre}[r] \oplus k$).
- Window property **monotonic** (adding elements only hurts / only helps) $\rightarrow$ **two pointers** instead of binary search per position.
- "Sum of f over ALL subarrays/pairs" $\rightarrow$ flip to **per-element contribution**: $a[i]$ sits in $(i+1) \cdot (n-i)$ subarrays; count how often each element/bit/pair is counted instead of iterating objects.
- "Count pairs (i, j) with condition" $\rightarrow$ sort + two pointers, or sweep left $\rightarrow$ right holding earlier elements in a Fenwick over values (inversions pattern).
- Answer is min/max cost with weird operations $\rightarrow$ hunt for an **invariant** first (sum mod k, parity, coloring) — it gives impossibility proofs AND constructions.
- Operations look irreversible $\rightarrow$ **reverse the process** (deletions become insertions, run queries backwards, last move first).
- Optimize over sequences with an exchangeable order $\rightarrow$ **sort by an exchange argument**: comparator "a before b iff ab-combined beats ba-combined" (classic: concatenate numbers into the largest string).

Arrays & queries

- Range ADD, values read only at the end $\rightarrow$ **difference array**: $d[1] += x$, $d[r+1] -= x$, prefix at the end. 2D: $\pm x$ at the 4 corners, 2D prefix.
- "Max number of overlapping intervals" $\rightarrow$ **events**: +1 at l, -1 at r+1, running maximum of the prefix sum.
- **# distinct values in range** without Mo: offline, sort queries by r; sweep r with a Fenwick: +1 at position of value, -1 at its previous occurrence; answer = sum over $[l, r]$.
- Online too hard? Almost every query problem collapses **offline**: sort queries by r / by value / by block and sweep. ("edges with weight $\leq X$ connect u,v?" $\rightarrow$ sort both, grow a DSU.)
- Frequencies + $\sqrt{v}$ threshold: values occurring $> \sqrt{v}$ times number $\leq \sqrt{v}$ ("heavy") — brute-force the heavy ones, bound the light ones per group.
- Divisor-style double loop for $d \{ \text{for } (j = d; j \leq n; j += d) \}$ is **$O(n \log n)$** , not $O(n^2)$ — harmonic series. Basis of many counting DPs.
- k-th smallest / rank queries on values up to $1e9 \rightarrow$ coordinate-compress, Fenwick over ranks, or ordered-multiset.

Graphs & trees

- **Subtree queries $\rightarrow$ Euler tour**: $\text{subtree}(u)$ is the contiguous range $[\text{in}[u], \text{in}[u] + \text{sz}[u] - 1]$ — Fenwick/segtree on the tour order (the HLD ordering gives this for free).
- Path ADD on a tree, read at the end $\rightarrow$ **tree difference**: +x at u and v, -x at lca, -x at $\text{par}[lca]$; final value of a node = sum over its subtree (one DFS).
- "Remove edges" / "destroy nodes" queries $\rightarrow$ process **in reverse** with DSU: deletions become additions.
- "Nearest special cell/node" for every position $\rightarrow$ **multi-source BFS** (push all specials at distance 0).
- Edge weights only 0/1 $\rightarrow$ **0-1 BFS** with a deque (push_front on 0-edges) — Dijkstra without the log.
- Collecting sets up a tree (colors in subtree, ...) $\rightarrow$ **small-to-large**: always merge the smaller set into the bigger; total $O(n \log^2 n)$.
- "Answer for EVERY node as root" $\rightarrow$ **rerooting**: one DFS computing down-answers, a second pushing the parent-side answer down (combine prefix/suffix over children).
- Each node has exactly one outgoing edge (functional graph) $\rightarrow$ it's cycles with trees hanging off; k-th successor via binary lifting like `lca.cpp`.

When stuck

- Write the **brute force anyway** ($n \leq 8$): stare at outputs for the pattern, then keep it for stress testing against the real solution with `Other/random.cpp`.
- Fix something and see what's left: the maximum element, the leftmost chosen index, the root — "enumerate the boss, optimize the rest".
- Constraints are a hint (rough map):
 - $n \leq 11 \rightarrow O(n!)$ · $n \leq 20 \rightarrow \text{bitmask } 2^n$ · $n \leq 40 \rightarrow$ meet in the middle ·
 - $n \leq 500 \rightarrow O(n^3)$ · $n \leq 5000 \rightarrow O(n^2)$ · $n \leq 2 \cdot 10^5 \rightarrow O(n \log n)$ ·
 - $n \leq 10^6 \rightarrow O(n)$ · $n \sim 10^9 \rightarrow O(\sqrt{n})$ / math · $n \sim 10^{18} \rightarrow O(\log n)$: binary search / matrix expo / digit DP.

Number Theory

divisors.cpp

NUMBER OF DIVISORS $d(n)$ + SUM OF DIVISORS $\sigma(n)$ — trial division, $O(\sqrt{n})$

RECALL — both come straight from the prime factorization $n = p_1^{e_1} \cdot p_2^{e_2} \cdot \dots$

$d(n) = (e_1+1) \cdot (e_2+1) \cdot \dots$ (pick each prime's exponent independently)

$\sigma(n) = \prod \text{over } p \text{ of } (1 + p + p^2 + \dots + p^e)$ (geometric sum per prime)

FACTS WORTH REMEMBERING:

- $d(n)$ is ODD $\iff$ n is a PERFECT SQUARE (divisors pair up except $\sqrt{n}$).
- $d(n)$ is small: max 1344 for $n \leq 1e9$, max 103680 for $n \leq 1e18$
- $\rightarrow$ "loop over all divisors" is usually fine after factorizing.
- many queries with $n < N$? factorize with spf from linear-sieve.cpp in $O(\log n)$ and apply the same formulas.
- $\sigma(n)$ can be $\sim 6x$ n — near 11's limit for $n \sim 1e18$, careful.
- perfect number: $\sigma(n) = 2n$.

```
long long numberOfDivisors(long long num) {
    long long total = 1;
    for (int i = 2; (long long)i * i <= num; i++) {
        if (num % i == 0) {
            int e = 0;
            do {
                e++;
                num /= i;
            } while (num % i == 0);
            total *= e + 1;                // exponent e contributes (e+1) choices
        }
    }
    if (num > 1) total *= 2;              // leftover prime > sqrt: exponent 1
    return total;
}

long long SumOfDivisors(long long num) {
    long long total = 1;
    for (int i = 2; (long long)i * i <= num; i++) {
        if (num % i == 0) {
            int e = 0;
            do {
                e++;
                num /= i;
            } while (num % i == 0);

            long long sum = 0, pow = 1;    // 1 + p + ... + p^e
            do {
                sum += pow;
                pow *= i;
            } while (e-- > 0);
            total *= sum;
        }
    }
    if (num > 1) total *= (1 + num);      // leftover prime p: (1 + p)
    return total;
}
```

euler-phi.cpp

EULER'S TOTIENT $\phi(n)$ — # of $1..n$ coprime with n

Formula: $\phi(n) = n * \prod_{\text{prime } p|n} (1 - 1/p)$

-> implemented as `res -= res/p` per distinct prime (order doesn't matter).

$\phi(\text{single } n)$: $O(\sqrt{n})$. ϕ for ALL $1..N$: sieve version, $O(N \log \log N)$.

THE FACTS YOU ACTUALLY USE:

- EULER'S THEOREM: $a^{\phi(m)} \equiv 1 \pmod{m}$ when $\gcd(a, m) = 1$.
(generalizes Fermat: $\phi(\text{prime}) = \text{prime}-1$)
- HUGE EXPONENT REDUCTION (power towers, exponent given as giant number):
 $a^b \equiv a^{(b \bmod \phi(m) + \phi(m))} \pmod{m}$ valid for $b \geq \log_2(m)$,
and this one works EVEN IF $\gcd(a, m) \neq 1$ (the $+\phi(m)$ is what makes it safe – don't drop it unless you know $\gcd = 1$!).
- ϕ is multiplicative: $\phi(a*b) = \phi(a)*\phi(b)$ when $\gcd(a,b) = 1$.
- sum of $\phi(d)$ over all divisors d of n = n .
- # of fractions k/n in lowest terms, cycle structures, "count coprime pairs".

```

ll phi(ll n) {                                // O(sqrt n), single value
    ll res = n;
    for (ll p=2; p*p<=n; p++) if (n % p == 0) {
        while (n % p == 0) n /= p;
        res -= res / p;                        // res *= (1 - 1/p)
    }
    if (n > 1) res -= res / n;                  // leftover prime factor > sqrt
    return res;
}

int phiAll[N];                                // phi for every 1..N-1 (sieve style)
void phiSieve() {
    iota(phiAll, phiAll+N, 0);
    for (int i=2; i<N; i++)
        if (phiAll[i] == i)                    // untouched -> i is prime
            for (int j=i; j<N; j+=i)
                phiAll[j] -= phiAll[j] / i;
}

```

ext-gcd-crt.cpp**EXTENDED GCD + MODULAR INVERSE (any mod) + CRT**

extgcd(a, b, x, y): returns $g = \gcd(a, b)$ and fills x, y with $a*x + b*y = g$.

WHAT IT UNLOCKS:

- 1) INVERSE MOD ANY m (m need NOT be prime, unlike Fermat in combinatorics.cpp):
 $\text{inv}(a, m)$ valid iff $\gcd(a, m) == 1$.
- 2) LINEAR DIOPHANTINE $a*x + b*y = c$:
 solvable iff $g \mid c$; scale: $x_0 = x*(c/g)$, $y_0 = y*(c/g)$;
 ALL solutions: $x = x_0 + t*(b/g)$, $y = y_0 - t*(a/g)$, t any integer
 (use that to shift x into a wanted range).
- 3) LINEAR CONGRUENCE $a*x \equiv b \pmod{m}$:
 solvable iff $g = \gcd(a, m)$ divides b; then $x \equiv (b/g)*\text{inv}(a/g, m/g) \pmod{m/g}$,
 i.e. g distinct solutions mod m.
- 4) CRT below.

CRT — combine $x \equiv r_1 \pmod{m_1}$ and $x \equiv r_2 \pmod{m_2}$ into one congruence.

GENERAL version: m_1, m_2 do NOT need to be coprime.

Returns {x, lcm(m_1, m_2)} with $0 \leq x < \text{lcm}$, or {-1, -1} if incompatible
 (possible only in the non-coprime case, when $r_1 \not\equiv r_2 \pmod{\gcd}$).

Fold a whole system by chaining: $\text{cur} = \text{crt}(\text{cur}, \text{next})$, ...

!! pass $0 \leq r < m$ (normalize with $((r \% m) + m) \% m$ first).

Overflow-safe up to $\text{lcm} \sim 9e18$ (`__int128` used at the one risky multiply).

```
ll extgcd(ll a, ll b, ll& x, ll& y) {
    if (!b) { x = 1; y = 0; return a; }
    ll x1, y1, g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

ll inv(ll a, ll m) {
    // inverse of a mod m — REQUIRES gcd(a, m) == 1
    ll x, y;
    extgcd(a, m, x, y);
    return ((x % m) + m) % m;
}

pair<ll, ll> crt(ll r1, ll m1, ll r2, ll m2) {
    ll x, y, g = extgcd(m1, m2, x, y); // m1*x + m2*y = g
    if ((r2 - r1) % g) return {-1, -1}; // no solution
    ll l = m1 / g * m2; // lcm
    ll d = m2 / g;
    ll k = (r2 - r1) / g % d * (x % d) % d; // how many m1-steps to add to r1
    if (k < 0) k += d;
    ll ans = r1 + (ll)((__int128)k * m1 % l); // k*m1 can pass 9e18 mid-multiply
    return {ans % l, l};
}
```

linear-sieve.cpp

LINEAR SIEVE — $O(N)$, gives SMALLEST PRIME FACTOR of every number

`spf[i]` = smallest prime factor of `i` (`spf[i] == i` $\Leftrightarrow$ `i` is prime)

`primes` = ALL primes $< N$ (complete list, unlike plain sieve variant).

WHY LINEAR: each composite `x` is crossed out EXACTLY ONCE, by (its smallest prime `p`) * (`x/p`) — that's what the `primes[j] <= spf[i]` cutoff guarantees.

THE KILLER FEATURE — factorize any `x < N` in $O(\log x)$:

```
while (x > 1) {
    int p = spf[x], cnt = 0;
    while (x % p == 0) x /= p, cnt++;
    // (p, cnt) is one prime-power of x
}
-> lets you factorize MANY numbers fast (vs trial division  $O(\sqrt{x})$  each).
```

ALSO GOOD FOR multiplicative functions (`phi`, `mobius`, `#divisors`) computed for ALL `i < N` inside this same loop — ask for the extended version if needed.

```
int spf[N];
vector<int> primes;

void LinearSieve() {
    for (int i=2; i<N; i++) {
        if (!spf[i]) {                // nothing crossed i yet -> prime
            spf[i] = i;
            primes.pb(i);
        }

        // cross i * p for primes p <= spf[i]; p becomes the smallest factor of i*p
        for (int j=0; j<primes.size() && i * primes[j]<N && primes[j] <= spf[i]; j++) {
            spf[i*primes[j]] = primes[j];
        }
    }
}
```

miller-rabin.cpp

MILLER-RABIN — deterministic primality test for ALL 64-bit numbers

isPrime(n) for n up to $\sim 1.8e19$ (unsigned 64-bit), $O(12 * \log n)$ mulmods.

Use when n is way too big for a sieve (e.g. "is $1e18 + 9$ prime?").

RECALL — the test with base a:

write $n-1 = d * 2^r$ (d odd). n passes base a if
 $a^d \equiv 1 \pmod{n}$ or $a^{d*2^i} \equiv n-1$ for some $0 \leq i < r$.
 Every prime passes every base; composites fail at least 3/4 of bases.
 With the FIXED base set {2,3,5,7,...,37} (first 12 primes) the test is
 PROVEN exact for all $n < 3.3e24$ -> covers the whole 64-bit range.
 (The old shorter set {2..13} is only safe below $3.47e12$ — a real WA trap!)

NOTES:

- mulmod uses `__uint128_t` -> GCC/Clang only (fine on Codeforces/ECPC judges).
- `millerRabin(n, a)` with $n \% a == 0$ answers directly (handles small $n / n == a$).
- Everything is unsigned long long (ull) — keep it that way, $n-1$ underflows otherwise... it doesn't since $n \geq 2$, but signed overflow in mulmod WOULD break, hence ull.

RELATED (not included): Pollard's rho for FACTORING 64-bit numbers — ask if needed.

```
ull mulmod(ull a, ull b, ull m) {
    return (__uint128_t)a * b % m;    // 128-bit intermediate, no overflow
}
ull powmod(ull a, ull b, ull m) {
    ull res = 1; a %= m;
    for (; b > 0; b >>= 1) {
        if (b & 1) res = mulmod(res, a, m);
        a = mulmod(a, a, m);
    }
    return res;
}
bool millerRabin(ull n, ull a) {      // does n pass witness a?
    if (n % a == 0) return n == a;    // shares a factor with a -> prime only if n == a
    ull d = n - 1; int r = 0;
    while (d % 2 == 0) d /= 2, r++;    // n-1 = d * 2^r
    ull x = powmod(a, d, n);
    if (x == 1 || x == n - 1) return true;
    for (int i = 0; i < r - 1; i++) {  // keep squaring, look for n-1
        x = mulmod(x, x, n);
        if (x == n - 1) return true;
    }
    return false;                    // composite for sure
}
bool isPrime(ull n) {
    if (n < 2) return false;
    if (n == 2 || n == 3 || n == 5 || n == 7) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    // first 12 primes -> deterministic for ALL n < 2^64
    for (ull a : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL})
        if (!millerRabin(n, a)) return false;
    return true;
}
```

sieve.cpp**SIEVE OF ERATOSTHENES — $O(N \log \log N)$**

prime[i] = is i prime; primes = list of all primes < N.

Call Sieve() once at startup.

NOTES:

- Inner loop starts at $i*i$: smaller multiples were already crossed by smaller primes.
i is ll precisely so $i*i$ can't overflow int when N is large ($N > \sim 46341$).
- WARNING: the outer loop stops at $i*i < N$, so `primes` only receives primes up to $\sqrt{N}$! prime[] itself IS correct for all $i < N$. If you need the full primes list, collect it afterwards:
for (int i=2; i<N; i++) if (prime[i]) primes.pb(i);
(or use linear-sieve.cpp, which collects all of them by construction)
- Rough counts: #primes below $1e6 \sim 78498$, below $1e7 \sim 620k$, below $1e8 \sim 5.7M$.

VARIANT — smallest prime factor / $O(\log)$ factorization: see linear-sieve.cpp.

VARIANT — count divisors / sum of divisors for all i: loop $j += i$ from i (not $i*i$)
and do $\text{cnt}[j]++$ — that harmonic double loop is $O(N \log N)$.

```
bool prime[N];
vector<int> primes;

void Sieve()
{
    fill(prime, prime+N, true);
    prime[0] = prime[1] = false;

    for (ll i=2; i*i<N; i++) {          // ll so i*i can't overflow
        if (prime[i]) {
            primes.pb(i);               // WARNING: only primes up to sqrt(N)! see header
            for (ll j=i*i; j<N; j+=i) {
                prime[j] = false;
            }
        }
    }
}
```

Combinatorics

combinatorics.cpp**COMBINATORICS (factorials + modular inverse)**

nCr / nPr in $O(1)$ per query after $O(n)$ precompute.

REQUIREMENT: mod must be PRIME (inverse uses Fermat: $a^{(mod-2)} = a^{-1}$).

If mod is NOT prime -> use Pascal's triangle (ncr-pascal.cpp)
or factor out common primes manually.

USAGE:

```
Combinatorics C;
C.Init(maxN, 1e9+7); // maxN = largest n you will ever ask about
C.nCr(n, r); C.nPr(n, r);
Also gives you safe modular ops: Add / Sub / Mul / Div / Pow / Inv.
```

RECALL - common formulas:

- Stars and bars: #solutions of $x_1 + \dots + x_k = n$ ($x_i \geq 0$) = $C(n+k-1, k-1)$
- Catalan(n) = $C(2n, n) / (n+1) = C(2n, n) - C(2n, n+1)$
(balanced brackets, binary trees, non-crossing paths)
- Multiset permutations: $n! / (c_1! * c_2! * \dots)$
- Hockey stick: $\sum_{i=r..n} C(i, r) = C(n+1, r+1)$

```
class Combinatorics
{
public:
    ll mod;
    vector<ll> fact, InvFact;

    ll Pow(ll a, ll b) // binary exponentiation,  $O(\log b)$ 
    {
        ll res = 1;
        a %= mod;
        while(b)
        {
            if(b&1) res = (res * a) % mod;
            a = (a * a) % mod;
            b >>= 1;
        }
        return res;
    }

    ll Inv(ll a) {return Pow(a, mod-2);} // ONLY valid for prime mod, gcd(a,mod)=1
    ll Add(ll a, ll b) {return ((a%mod)+(b%mod))%mod;}
    ll Mul(ll a, ll b) {return ((a%mod)*(b%mod))%mod;}
    ll Sub(ll a, ll b) {return ((a-b)%mod + mod)%mod;} // handles negatives correctly
    ll Div(ll a, ll b) {return Mul(a, Inv(b));}

    void Init(ll n, ll x) // call ONCE with the max n needed;  $O(n + \log \text{mod})$ 
    {
        fact.resize(n+1); InvFact.resize(n+1);
        mod = x;

        fact[0] = 1;
        for(int i=1; i<=n; i++) fact[i] = Mul(fact[i-1], i);

        // one Pow call, then walk down: InvFact[i] = InvFact[i+1] * (i+1)
        InvFact[n] = Inv(fact[n]);
        for(int i=n-1; i>=0; i--) InvFact[i] = Mul(InvFact[i+1], i+1);
    }

    ll nPr(ll n, ll r) // ordered pick:  $n! / (n-r)!$ 
    {
        if(n < 0 || r < 0 || n < r) return 0; // out-of-range = 0, so you can call it blindly
        return Mul(fact[n], InvFact[n-r]);
    }

    ll nCr(ll n, ll r) // unordered pick:  $n! / (r! (n-r)!)$ 
    {
        if(n < 0 || r < 0 || n < r) return 0;
        return Mul(nPr(n, r), InvFact[r]);
    }
};
```

euler-pentagonal-partitions.cpp

PARTITION FUNCTION $p(n)$ — Euler's Pentagonal Number Theorem

$p(n)$ = number of ways to write n as a sum of positive integers, ORDER DOESN'T MATTER.

e.g. $p(4) = 5$: 4, 3+1, 2+2, 2+1+1, 1+1+1+1

RECURRENCE (this is the whole theorem):

$p(n) = \text{sum over } k=1,2,3,\dots \text{ of } (-1)^{(k-1)} * [p(n - g_1(k)) + p(n - g_2(k))]$
 where $g_1(k) = k(3k-1)/2$ and $g_2(k) = k(3k+1)/2$ (generalized pentagonal numbers:
 1, 2, 5, 7, 12, 15, 22, 26, ...). Stop when $g_1(k) > n$.
 Signs alternate in PAIRS: +, +, -, -, +, +, ... (that's the $k \& 1$ check below)

COMPLEXITY: $O(n * \sqrt{n})$ — only $O(\sqrt{n})$ pentagonal numbers are $\leq n$.

$n = 1e5 \rightarrow \sim 3e7$ ops, fine.

This is a SNIPPET (goes inside main/solve). Needs: n , mod .

Base case $p(0) = 1$ (the empty sum).

```
vector<ll> p(n+1);
p[0] = 1;
for (int i=1; i<=n; i++) {
    for (int k=1; ; k++) {
        ll g1 = 1LL * k * (3 * k - 1) / 2; // pentagonal number
        ll g2 = 1LL * k * (3 * k + 1) / 2; // its pair

        if (g1 > i) break; // both too big -> done with i

        ll term = p[i - g1];
        if (g2 <= i) term = (term + p[i - g2]) % mod;

        if (k & 1) p[i] = (p[i] + term) % mod; // odd k -> add
        else p[i] = (p[i] - term + mod) % mod; // even k -> subtract
    }
}
```

ncr-pascal.cpp**nCr VIA PASCAL'S TRIANGLE**

$c[n][r] = C(n, r) \% \text{mod}$, built with $c[n][r] = c[n-1][r] + c[n-1][r-1]$.

WHEN to prefer this over the factorial version (combinatorics.cpp):

- mod is NOT prime (no modular inverse needed here!)
- or you want plain (unmodded) values for small n (drop the % mod, use ll, n <= 66 fits ll)

COST: $O(N^2)$ time AND memory -> N up to ~3000-4000 max

(ll c[N][N] with N=4000 is already ~128 MB; use int if mod < 2^{31} to halve it).

NOTES:

- Entries with $r > n$ stay 0 (globals are zero-initialized) -> safe to read.
- Call pascal() once at startup.

```
ll c[N][N];
```

```
void pascal()
```

```
{
    for (int i=0; i<N; i++) {
        c[i][0] = c[i][i] = 1;           // C(n,0) = C(n,n) = 1
        for (int j=1; j<i; j++) {
            c[i][j] = (c[i-1][j] + c[i-1][j-1]) % mod; // take or skip the last element
        }
    }
}
```

DP

digit-dp.cpp

DIGIT DP — count numbers in a range with a digit property, $O(\text{len} * \text{STATES} * 10)$

THE FRAME (memorize this, it never changes):

```
answer for [L, R] = f(R) - f(L-1)    <- reduces everything to f(X) = count in [0, X]
go(pos, state, tight, started):
    pos      = which digit of X we're placing (left to right)
    tight    = are we still GLUED to X's prefix? (if yes, digit can't exceed X's digit;
               once we place something smaller, tight becomes false FOREVER -> free digits)
    started  = have we placed a nonzero digit yet? (false = still in leading zeros;
               matters when the property cares about actual digits, e.g. "no repeated
               digit", where leading zeros must NOT count as the digit 0)
    state    = THE problem-specific part (example below: digit sum mod K)
```

MEMOIZE only (!tight && started) states: tight paths are a single chain down X (visited once anyway) and not-started is a single all-zeros chain. So dp[][] needs no tight/started dimensions -> small and simple.

EXAMPLE WIRED IN: how many numbers in [0, X] have digit sum divisible by K.

ADAPT: replace `state`'s meaning + the base-case check + the transition line.

Common states: sum mod K, last digit (adjacent constraints), bitmask of used digits, "contains 13" flag, count of some digit — anything small.

PITFALLS:

- memset dp to -1 INSIDE f(X) (each X is a fresh memo — state meanings collide).
- f(L-1) with L = 0 -> f(-1): must return 0 (guard below).
- the empty/never-started path represents the number 0 — decide if the base case should count it (here: yes, sum 0 divides K) and mind it when L = 0.

```
string s;                // X as a string of digits
int K;                   // example parameter
ll dp[20][200];          // dp[pos][state] for the (!tight && started) universe

ll go(int pos, int state, bool tight, bool started) {
    if (pos == sz(s)) return state == 0;          // CHANGE: does `state` satisfy the property?

    if (!tight && started && dp[pos][state] != -1) return dp[pos][state];

    ll res = 0;
    int lim = tight ? s[pos] - '0' : 9;           // glued -> can't exceed X's digit
    for (int dig=0; dig<=lim; dig++) {
        int nstate = (state + dig) % K;           // CHANGE: the transition
        // if the property ignores leading zeros, guard with: started || dig > 0 ? ... : state
        res += go(pos+1, nstate, tight && dig == lim, started || dig > 0);
    }

    if (!tight && started) dp[pos][state] = res;
    return res;
}

ll f(ll x) {
    // count of valid numbers in [0, x]
    if (x < 0) return 0;
    s = to_string(x);
    memset(dp, -1, sizeof dp);
    return go(0, 0, true, false);
}
// answer for [L, R] = f(R) - f(L-1)
```

lis.cpp

LIS — Longest Increasing Subsequence, $O(n \log n)$, with full reconstruction

RECALL — the invariant: $d[\text{len}-1]$ = the SMALLEST possible tail value of an increasing subsequence of length len . d is always sorted, so $a[i]$ can extend the longest length whose tail is $< a[i]$ — found by binary search; $a[i]$ then becomes a better (smaller) tail for that length.
(d is NOT an actual subsequence! reconstruction needs the $\text{par}[]$ bookkeeping.)

>>> THE ONE SWITCH <<<

STRICTLY increasing : lower_bound (as written)
NON-DECREASING (ties ok): upper_bound — that's the entire change
Longest strictly DECREASING: run on negated values (or reversed array).

SNIPPET (inside solve). Needs n , a . Produces:

```
sz(d)    = LIS length
lis      = one actual LIS (values; use indices via par-chain if needed)
lenAt[i] = length of the best increasing subsequence ENDING at i (often useful alone)
```

```
vector<int> d; // d[len-1] = best (smallest) tail for that len
vector<int> lenAt(n), par(n), idxOfLen; // idxOfLen[len-1] = index holding that tail
for (int i=0; i<n; i++) {
    int j = lower_bound(all(d), a[i]) - d.begin(); // upper_bound -> non-decreasing
    if (j == sz(d)) d.pb(a[i]), idxOfLen.pb(i); // extends the longest -> new length
    else d[j] = a[i], idxOfLen[j] = i; // better (smaller) tail for length j+1
    lenAt[i] = j + 1;
    par[i] = j ? idxOfLen[j-1] : -1; // who comes before me in MY subsequence
}

vector<int> lis; // reconstruct by walking parents
for (int cur = idxOfLen.back(); ~cur; cur = par[cur]) lis.pb(a[cur]);
reverse(all(lis));
```

matrix-exponentiation.cpp**MATRIX EXPONENTIATION — linear recurrences in $O(k^3 \log b)$**

WHEN: $f(n)$ depends linearly on the previous k terms and n is HUGE ($1e18$).

$f(n) = c_1 * f(n-1) + c_2 * f(n-2) + \dots + c_k * f(n-k) \rightarrow \text{answer} = T^n * \text{base}$

RECIPE (Fibonacci as the example, $f(0)=0, f(1)=1$):

$T = \begin{Bmatrix} 1 & 1 \\ 1 & 0 \end{Bmatrix}$, // $\begin{bmatrix} f(n+1) & f(n) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} f(n) \\ f(n-1) \end{bmatrix}$

$\text{auto } Tn = \text{binPow}(T, n)$; // T^n

$f(n) = Tn[1][0] * f(1) + Tn[1][1] * f(0)$ (or read off $Tn[0][1] = f(n)$ directly for fib)

General k : first row = the coefficients $c_1..c_k$, subdiagonal = 1s (shift register).

ALSO WORKS FOR: # of paths of length EXACTLY n in a graph (T = adjacency matrix!),
probabilities/expected values with fixed transitions, "DP with tiny state, giant n ".

!! PITFALL: binPow takes `a` BY REFERENCE and DESTROYS it (squares it in place).

Pass a copy if you need the matrix again: `auto tmp = T; auto Tn = binPow(tmp, n);`

Feasibility: $k^3 * \log(n) \rightarrow k=100, n=1e18$ is $\sim 6e7$ mults, OK.

```
vector<vector<int>> mult(vector<vector<int>>& a, vector<vector<int>>& b) {
    const int n = a.size(), k = a[0].size(), m = b[0].size(); // (n x k) * (k x m)
    vector<vector<int>> ret(n, vector<int>(m));
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            for (int z=0; z<k; z++) {
                ret[i][j] = (ret[i][j] + 1LL * a[i][z] * b[z][j] % mod) % mod;
            }
        }
    }
    return ret;
}

vector<vector<int>> binPow(vector<vector<int>>& a, ll b) { // !! modifies a (see header)
    const int n = a.size(), m = a[0].size();
    vector<vector<int>> ret(n, vector<int>(m));
    for (int i=0; i<n; i++) ret[i][i] = 1; // identity matrix

    while (b) {
        if (b&1) ret = mult(ret, a);
        a = mult(a, a);
        b >>= 1;
    }

    return ret;
}
```

rerooting.cpp

REROOTING — answer "as if rooted at u" for EVERY u, $O(n)$ total

Two DFS passes: (1) normal tree DP computing $down[u]$ = answer inside u's subtree; (2) push a parent-side value $up[v]$ down the tree, combining what the parent knows MINUS v's own contribution.

As written: $ans[u]$ = SUM of distances from u to every node (classic).

down pass: $sz[u]$, $down[u]$ = sum of dist to nodes in u's subtree

push down: $ans[v] = ans[u] - sz[v] + (n - sz[v])$

(crossing edge u-v: v's subtree gets 1 closer, the rest 1 farther)

RECALL — adapting it:

- Works directly whenever the parent-side value is computable by "whole answer at parent MINUS child's contribution" (sum-like combines).
- Combine NOT invertible (max depth, gcd, ...)? Then for each u build PREFIX and SUFFIX combines over its children's values; child v gets $pref[i-1] + suf[i+1] + \text{parent's up-value}$. Same $O(n)$, just more bookkeeping.
- Values on EDGES: fold the edge weight where the +1s are.

namespace Reroot

```
{
    int n; vector<vector<int>> adj;
    vector<ll> down, ans; vector<int> sz;

    void dfs1(int u, int p) {
        sz[u] = 1, down[u] = 0;
        for (int v : adj[u]) if (v != p) {
            dfs1(v, u);
            sz[u] += sz[v];
            down[u] += down[v] + sz[v]; // every node in v's subtree is 1 farther
        }
    }

    void dfs2(int u, int p) {
        for (int v : adj[u]) if (v != p) {
            // reroot u -> v: v's subtree (sz[v] nodes) gets closer, rest gets farther
            ans[v] = ans[u] - sz[v] + (n - sz[v]);
            dfs2(v, u);
        }
    }

    void solve() {
        // fill n and adj first (0-indexed)
        sz.assign(n, 0), down.assign(n, 0), ans.assign(n, 0);
        dfs1(0, -1);
        ans[0] = down[0];
        dfs2(0, -1); // now ans[u] is correct for every u
    }
}
```

sos-dp.cpp

SOS DP (sum over subsets) — $f[m]$ = sum of $a[s]$ over all submasks s of m

$O(B * 2^B)$ for B bits — beats the naive 3^B submask loop when you need ALL masks.

Think of it as a B -dimensional prefix sum: one bit-dimension per pass.

USAGE: load point values into f , run the loop, read answers from f (in-place).

RECALL:

- SUPERSET sums: flip the condition — add $f[m \mid 1 \ll b]$ when bit b is NOT set.
- Inverse (recover point values from submask sums / do exact-cover inclusion-exclusion): same loops, SUBTRACT instead of add.
- "count pairs with $a \& b == 0$ ": b must be a submask of $\sim a \rightarrow$ SOS over complements. " $a \& b == a$ " pairs $\rightarrow$ superset sums.
- Single mask only? enumerate its submasks directly in $O(2^{\text{popcount}})$:
 for (int $s = m$; s ; $s = (s - 1) \& m$) { ... use s ... } // add $s=0$ if needed
 Over ALL m together that's the 3^B total — fine for $B \leq \sim 20$ offline.
- max/min instead of +: works the same (it's just a different monoid).

namespace SOS

```
{
    int B; // number of bits, masks are 0 .. 2^B - 1
    vector<ll> f; // in: f[mask] = point value; out: f[mask] = sum over submasks

    void sos() {
        for (int b = 0; b < B; b++)
            for (int m = 0; m < (1 << B); m++)
                if (m >> b & 1) f[m] += f[m ^ (1 << b)];
        // superset version: if (!(m >> b & 1)) f[m] += f[m | (1 << b)];
        // inverse: same loops, f[m] -= ... (undoes one bit-dimension per pass)
    }
}
```

tree-knapsack.cpp

TREE KNAPSACK / "merging subtree DPs" — $O(n^2)$ total (NOT n^3 !)

$dp[u][k]$ = best total value of a CONNECTED subgraph that contains u ,
 stays inside u 's subtree, and has exactly k vertices.
 (connected + contains $u \Leftrightarrow$ "u plus some children subtree-pieces")

RECALL — the merge (it's just 0/1 knapsack over children):

```
process child v: dp[u][i + j] <- dp[u][i] + dp[v][j]
- i loops DOWNWARD over what u had BEFORE this child -> each child used once
  (same reason 0/1 knapsack loops capacity downward)
- loops are capped by size[u]-so-far and size[v]; that cap is the whole magic:
  each PAIR of vertices (x in earlier part, y in v) is counted once, at their LCA
  -> total work = #pairs =  $O(n^2)$ . NEVER loop to n — that makes it  $O(n^3)$ .
```

SNIPPET (inside solve). Needs: n , adj , a (node values), INF ($\sim 1e18$ with ll dp).

-INF marks "impossible size". Answer: max over $dp[0][k]$ (root=0) for the k you need;
 for "best anywhere in the tree", also consider $dp[u][k]$ at every u — any connected
 subgraph is captured at its TOPMOST node.

VARIANTS with the same skeleton: "keep k edges/vertices, maximize", counting
 versions (sum instead of max, mod), costs on edges (add edge cost when $j > 0$).

```
vector<vector<ll>> dp(n, vector<ll>(n+1, -INF));
vector<int> size(n);
function<void(int,int)> dfs = [&](int u, int p) {
    dp[u][1] = a[u]; // just u itself
    size[u] = 1;
    for (int v : adj[u]) if (v != p) {
        dfs(v, u);

        // knapsack-merge v into u; i DOWNWARD, both loops size-capped (see header)
        for (int i=size[u]; i;i--) {
            for (int j=1; j<=size[v]; j++) {
                if (dp[u][i] == -INF || dp[v][j] == -INF) continue;
                dp[u][i+j] = max(dp[u][i+j], dp[u][i] + dp[v][j]);
            }
        }
        size[u] += size[v];
    }
};
dfs(0, -1);
```

Graphs & Trees

articulation-points.cpp

ARTICULATION POINTS (cut vertices) — Tarjan, $O(n + m)$

Cut vertex = node whose REMOVAL (with its edges) disconnects the graph.

RECALL — same in/low machinery as bridges, DIFFERENT condition:

non-root u is a cut vertex $\Leftrightarrow$ SOME child v has $\text{low}[v] \geq \text{in}[u]$
 (v 's subtree can't climb strictly ABOVE $u \rightarrow$ removing u strands it)
 root is special: cut vertex $\Leftrightarrow$ it has ≥ 2 DFS CHILDREN.
 $\ggg$ compare: bridge condition was $\text{low}[v] > \text{in}[u]$ (strict!) on the EDGE. $\lll$

NOTES:

- parallel edges DON'T matter here (unlike bridges): removing the VERTEX kills all copies anyway, so skipping the parent by node is fine.
Self-loops: just don't add them to adj.
- a graph can have 0 cut vertices (biconnected) — e.g. a simple cycle.
- one node can be "the" cut vertex for many components; count via how many children satisfy the condition (+1 for the rest of the graph) if a problem asks "into how many pieces does removing u split the graph".

CALL: for (int i=0; i<n; i++) if (!in[i]) dfsCut(i, -1);

```
int n, timer = 1, in[N], low[N];
bool isCut[N];
vector<int> adj[N];

void dfsCut(int u, int p) {
    in[u] = low[u] = timer++;
    int children = 0;
    for (int v : adj[u]) if (v != p) {
        if (in[v])
            low[u] = min(low[u], in[v]);           // back edge
        else {
            dfsCut(v, u); children++;
            low[u] = min(low[u], low[v]);
            if (p != -1 && low[v] >= in[u])         // v's subtree is trapped under u
                isCut[u] = true;
        }
    }
    if (p == -1 && children > 1) isCut[u] = true; // root rule
}
```

bellman-ford.cpp

BELLMAN-FORD — shortest paths WITH negative edges + negative cycle detection, $O(n*m)$

RECALL: relax every edge, $n-1$ rounds. A shortest path has $\leq n-1$ edges, so after round k all shortest paths using $\leq k$ edges are final. If ROUND n STILL relaxes something $\rightarrow$ a negative cycle is reachable from src.

USAGE:

```
vector<ll> dist;
bool hasNegCycle = bellmanFord(n, src, edges, dist);
dist[u] == INF  $\rightarrow$  unreachable.
```

!! the `dist[u] != INF` guard is mandatory: without it $INF + (\text{negative } w)$ "improves" unreachable nodes and everything is garbage.

NOTES:

- returns true = SOME negative cycle reachable from src. Nodes whose real distance is $-\infty$: collect every v relaxed in the extra pass, then BFS/DFS forward from them – everything visited has $dist = -\infty$.
- negative cycle ANYWHERE in the graph (no source): init ALL $dist$ to 0 (equivalent to a virtual source with 0 -edges to everyone) and run the same loop.
- need the cycle itself: remember $par[v]$ on relaxations; from a node relaxed in the extra pass, follow par n times (guaranteed to land ON the cycle), then loop.

```
struct Edge { int u, v; ll w; };
```

```
bool bellmanFord(int n, int src, vector<Edge>& edges, vector<ll>& dist) {
    dist.assign(n, INF);
    dist[src] = 0;

    for (int it=0; it<n-1; it++) {
        bool any = false;
        for (auto& [u, v, w] : edges)
            if (dist[u] != INF && dist[u] + w < dist[v])
                dist[v] = dist[u] + w, any = true;
        if (!any) break; // already stable  $\rightarrow$  no cycle possible either
    }

    for (auto& [u, v, w] : edges) // round n: anything still moving = neg cycle
        if (dist[u] != INF && dist[u] + w < dist[v])
            return true;
    return false;
}
```

bridges-using-tarjan.cpp

BRIDGES (Tarjan, one DFS, $O(n + m)$)

Bridge = edge whose removal disconnects the graph.

RECALL — how it works:

in[u] = DFS discovery time (also serves as "visited": 0 = unvisited, so timer starts at 1)
 low[u] = smallest discovery time reachable from u's subtree using tree edges
 + AT MOST ONE back edge.
 Edge (u -> child v) is a bridge $\Leftrightarrow$ low[v] > in[u]
 (v's subtree has NO back edge climbing above v, so cutting the edge separates it.)

SETUP:

adj[u] holds {neighbor, edge id}. Edge id needed because we skip the PARENT EDGE,
 not the parent node -> parallel edges (u-v twice) are handled correctly:
 a doubled edge is NOT a bridge, and skipping by node would wrongly report it as one.

CALL: for (int i=0; i<n; i++) if (!in[i]) tarjan(i, -1); // graph can be disconnected

READ: isBridge[e] per edge id.

NOTE: recursive — for $n \sim 1e5+$ on small stack limits, convert to iterative
 or use the lambda version (see bridge-tree.cpp).

```
vector<pair<int,int>> adj[N]; // {v, edge id}
int n, m, timer = 1, in[N], low[N], isBridge[N];

void tarjan(int u, int pe) { // pe = id of the edge we came from
    in[u] = low[u] = timer++;
    for (const auto& [v, e] : adj[u]) if (e != pe) { // skip parent EDGE (not node!)
        if (in[v])
            low[u] = min(low[u], in[v]); // back edge -> can climb to v's time
        else {
            tarjan(v, e);
            low[u] = min(low[u], low[v]); // child's reach is our reach
            if (low[v] > in[u]) // child can't climb above u -> bridge
                isBridge[e] = true;
        }
    }
}
```

bridge-tree.cpp

BRIDGE TREE (a.k.a. 2-edge-connected component tree), $O(n + m)$

PIPELINE (3 steps below):

- 1) Mark all bridges (Tarjan – see bridges-using-tarjan.cpp for the theory).
- 2) Flood-fill WITHOUT crossing bridges -> $\text{comp}[u]$ = 2-edge-connected component id.
(inside a component, every pair of nodes has 2 edge-disjoint paths)
- 3) Every bridge connects two different components -> those are the TREE edges.

RESULT: `tree` = the bridge tree on `id` nodes. Any cycle of the original graph is squeezed inside a single tree node, and every original bridge = a tree edge.

WHY YOU WANT IT — turns "edge-connectivity" problems into TREE problems:

- # bridges on the path $u \rightarrow v$ = $\text{tree-distance}(\text{comp}[u], \text{comp}[v])$
- min edges to add to make the whole graph 2-edge-connected = $(\text{leaves} + 1) / 2$
- "does removing edge e disconnect u from v " -> is e a bridge on their tree path

SNIPPET (inside main). Nodes 1-indexed in input, converted to 0-indexed here.

Parallel edges are fine (parent skipped by edge id `pe`, not by node).

```
int n, m; cin >> n >> m;
vector<vector<pair<int,int>>> adj(n); // {v, edge id}
vector<pair<int,int>> edges(m);
for (int i=0; i<m; i++) {
    int u, v; cin >> u >> v;
    adj[--u].emplace_back(--v,i);
    adj[v].emplace_back(u,i);
    edges[i] = {u,v};
}

// ----- step 1: mark bridges -----
int timer = 0;
vector<bool> isBridge(m);
vector<int> in(n,-1), low(n); // in = -1 means unvisited
function<void(int,int)> tarjan = [&](int u, int pe) {
    in[u] = low[u] = timer++;
    for (const auto& [v, e] : adj[u]) if (e != pe) {
        if (~in[v])
            low[u] = min(low[u], in[v]); // back edge
        else {
            tarjan(v, e);
            low[u] = min(low[u], low[v]);
            if (low[v] > in[u]) isBridge[e] = true;
        }
    }
};
tarjan(0,-1); // if the graph may be disconnected: loop over all unvisited nodes

// ----- step 2: components = flood fill that never crosses a bridge -----
vector<int> comp(n,-1); int id = 0;
function<void(int)> setComp = [&](int u) {
    comp[u] = id;
    for (const auto& [v, e] : adj[u]) if (!isBridge[e] && comp[v] == -1) {
        setComp(v);
    }
};
for (int i=0; i<n; i++) {
    if (comp[i] == -1) {
        setComp(i);
        id++;
    }
}

// ----- step 3: bridges become the tree edges -----
vector<vector<int>> tree(id);
for (const auto& [u, v] : edges) {
    int cu = comp[u], cv = comp[v];
    if (cu != cv) { // endpoints in different comps <=> this edge is a bridge
        tree[cu].pb(cv);
        tree[cv].pb(cu);
    }
}
// now run LCA / DFS / DP on `tree`; map original node u -> tree node comp[u]
```

dijkstra.cpp

DIJKSTRA — single-source shortest paths, $O((n + m) \log n)$

!! ONLY for NON-NEGATIVE edge weights. One negative edge -> use bellman-ford.cpp.

USAGE: build adj as {v, w} pairs; dijkstra(src, n); read dist[] / rebuild path via par[].
dist[u] == INF -> unreachable.

RECALL — lazy deletion: we push duplicates into the pq instead of decreasing keys.
The line 'if (d > dist[u]) continue;' throws away stale entries — FORGETTING IT
is the classic bug (turns the algorithm exponential on some tests).

VARIANTS:

- multi-source ("nearest shop"): push ALL sources with dist 0 before the loop.
- weights only 0/1 -> 0-1 BFS with a deque (push_front on 0-edges), $O(n+m)$.
- "k-th cheapest path" / state graphs: node = (vertex, state), same code.
- path RECONSTRUCTION: walk par[] back from target, reverse.

```
vector<pair<int,ll>> adj[N];           // {neighbor, weight}
ll dist[N];
int par[N];

void dijkstra(int src, int n) {
    fill(dist, dist+n, INF);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<>> pq; // min-heap {dist, node}
    dist[src] = 0; par[src] = -1;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // stale entry — DO NOT DELETE THIS LINE
        for (auto& [v, w] : adj[u]) {
            if (d + w < dist[v]) {
                dist[v] = d + w;
                par[v] = u;
                pq.push({dist[v], v});
            }
        }
    }
}
```

dsu.cpp

DSU / UNION-FIND (path compression + union by size)

Maintains disjoint sets under "merge two sets" + "which set is u in".

Both ops amortized $\sim O(1)$ (inverse Ackermann).

USAGE:

```
init(n);           // MUST call first (and again for every test case!)
find(u)            // representative of u's set; find(u)==find(v) <=> same set
merge(u, v)        // returns false if already in the same set
len[find(u)]       // size of u's component (only valid at the ROOT!)
```

RECALL — classic uses:

- Kruskal MST: sort edges by weight, add edge iff merge(u,v) succeeds.
- Count components: components -= merge(u,v) succeeded.
- Offline "connectivity after adding edges" queries.
- Cycle detection in undirected graph: merge fails -> edge closes a cycle.
- Bipartite / parity trick: node u -> two nodes (u, u+n) "u is white / u is black".

namespace DSU

```
{
    int sz; vector<int> par, len;    // par = parent pointer, len = component size (valid at root)

    void init(int n)
    {
        sz = n+3;
        par.assign(sz, 0), len.assign(sz, 1);
        iota(all(par), 0);          // everyone is their own parent
    }

    int find(int u) {
        // path compression: hang u (and the whole chain) directly off the root
        return par[u] == u ? u : par[u] = find(par[u]);
    }

    bool merge(int u, int v) {
        u = find(u), v = find(v);
        if (u == v) return false;    // already connected
        if (len[u] < len[v]) swap(u, v); // union by size: small tree under big tree
        par[v] = u, len[u] += len[v];
        return true;
    }
}
using namespace DSU;
```

dsu-rollback.cpp

DSU WITH ROLLBACK — undo merges in reverse order, $O(\log n)$ per op

!! NO path compression (it can't be undone cheaply) — that's why find is

$O(\log n)$, NOT $\sim O(1)$. Union by size keeps the tree shallow; DON'T "optimize".

!! Rollback is LIFO only: you can undo the LAST merges, not an arbitrary one.

USAGE:

```
init(n);
merge(u, v);           // as usual (failed merges are fine)
int t = snapshot();    // remember a point in time
... more merges ...
rollback(t);           // undo everything after the snapshot
```

RECALL — where this shows up:

- Mo with rollback (mo-with-rollback.cpp): add left-side elements, answer, undo.
- OFFLINE dynamic connectivity ("edge exists during time interval $[l, r]$ "): put each edge on the $O(\log q)$ nodes of a segment tree over TIME covering its interval; DFS the tree merging that node's edges, answer queries at leaves, rollback(t) when leaving the node. Handles deletions with a DSU!
- Bipartiteness / parity under edge insert+undo: same trick, store parity too.

namespace DSU

```
{
    vector<int> par, len;
    vector<pair<int,int>> hist;           // (root that got absorbed, root it went under)

    void init(int n) {
        par.assign(n+3, 0), len.assign(n+3, 1);
        iota(all(par), 0);
        hist.clear();
    }

    int find(int u) {                     // NO compression — walk up every time
        while (par[u] != u) u = par[u];
        return u;
    }

    bool merge(int u, int v) {
        u = find(u), v = find(v);
        if (u == v) return false;        // failed merges push nothing (nothing to undo)
        if (len[u] < len[v]) swap(u, v);
        par[v] = u, len[u] += len[v];
        hist.push_back({v, u});
        return true;
    }

    int snapshot() { return hist.size(); }
    void rollback(int t) {                // undo until only t merges remain
        while ((int)hist.size() > t) {
            auto [v, u] = hist.back(); hist.pop_back();
            par[v] = v, len[u] -= len[v];
        }
    }
}
```

using namespace DSU;

floyd-warshall.cpp

FLOYD-WARSHALL — ALL-pairs shortest paths, $O(n^3)$, n up to ~400-500

Handles negative EDGES fine; detects negative CYCLES (see below).

RECALL — $d[i][j]$ after iteration k = shortest $i \rightarrow j$ path using only intermediate vertices from $\{0..k\}$. >>> k MUST be the OUTERMOST loop <<< (k inside i/j is the classic silent-wrong-answer bug).

SETUP (snippet, inside solve):

```
d[i][i] = 0; d[u][v] = min edge weight (parallel edges: take min!); else INF.
```

NOTES:

- the INF guards below avoid $INF + w$ overflow/poisoning; alternatively use $INF = 1e18/2$ and clamp, but the guards are cleaner.
- NEGATIVE CYCLE: after the loops, $d[x][x] < 0$ for some $x \Leftrightarrow$ neg cycle. Pair (i,j) has dist $-\infty$ iff it goes through one:
exists x with $d[i][x] \neq INF \ \&\& \ d[x][j] \neq INF \ \&\& \ d[x][x] < 0$.
- path reconstruction: keep $nxt[i][j]$ = first hop of best $i \rightarrow j$; on improve: $nxt[i][j] = nxt[i][k]$; walk nxt from i until j .
- also good for transitive closure (bitset version) and min/max bottleneck ("minimize the maximum edge"): replace $+$ with $\max$ and $\min$ stays $\min$.

```
ll d[MAXN][MAXN]; // MAXN ~ 500 -> 2 MB of ll, fine
```

```
void floyd(int n) {
    for (int k=0; k<n; k++)
        for (int i=0; i<n; i++) {
            if (d[i][k] == INF) continue; // nothing goes through k from i
            for (int j=0; j<n; j++) {
                if (d[k][j] == INF) continue;
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
            }
        }
}
```

hld.cpp

HEAVY-LIGHT DECOMPOSITION (HLD)

Answer PATH queries (sum/min/max on the path $u..v$) with a segment tree, and support point/range updates. Each query touches $O(\log n)$ chains, so total $O(\log^2 n)$ per query with a segtree on top.

RECALL — the idea:

- $big[u]$ = "heavy child" = child with the largest subtree.
- $decompose()$ writes nodes in an order where every heavy chain is CONTIGUOUS, $in[u]$ = position of node u in that order, $head[u]$ = top node of u 's chain.
- Walking $u \rightarrow$ root switches chains only $O(\log n)$ times, because every time you leave a chain top, the subtree size at least doubles.

SETUP (order matters):

- 1) build adj, choose root r (usually 0)
- 2) $dfs0(r, -1);$ // sizes, heavy child, par, lvl
- 3) $timer = 0; head[r] = r; decompose(r, -1);$
- 4) build a segtree of size n ; put value of node u at position $in[u]$.
 $tree[pos]$ tells you WHICH node sits at segtree position pos (for initial build).

QUERY PATH $u..v$:

for (auto $[l, r] : getRanges(u, v)$) $ans = merge(ans, seg.query(l, r));$
 NOTE: the ranges come in arbitrary order (we jump from both ends) $\rightarrow$
 fine for COMMUTATIVE ops (sum/min/max/gcd). For non-commutative (e.g. hashing along the path) you must collect and order them carefully.

EDGE-WEIGHTED version (costs on edges, not nodes) — 2 changes:

- 1) store each edge's weight on the DEEPER endpoint (child), root holds identity
- 2) in $getRanges$, the shared-chain range must EXCLUDE the LCA itself:
 use the commented ``in[u]+1`` line instead (LCA's slot holds the edge ABOVE lca, which is not on the path).

BONUS TRICK (free with this ordering): the subtree of u is the contiguous range

$[in[u], in[u] + sz[u] - 1]$ $\rightarrow$ subtree queries/updates with the same segtree!

Multi-test: reset $timer = 0$ (and clear adj).

```
int n, timer, sz[N], in[N], tree[N], big[N], head[N], par[N], lvl[N];
vector<int> adj[N];

// sizes, heavy child, parent, level
void dfs0(int u, int p) {
    sz[u] = 1;
    big[u] = -1;
    for (int v : adj[u]) if (v != p) {
        par[v] = u;
        lvl[v] = lvl[u] + 1;
        dfs0(v, u);
        sz[u] += sz[v];
        if (big[u] == -1 || sz[v] > sz[big[u]]) big[u] = v; // heaviest child
    }
}

// write nodes to positions: heavy chain first => chain is contiguous in `in` order
void decompose(int u, int p) {
    tree[timer] = u; // tree[pos] = node at segtree position pos
    in[u] = timer++;
    if (~big[u]) {
        head[big[u]] = head[u]; // heavy child continues MY chain
        decompose(big[u], u);
    }

    for (int v : adj[u]) if (v != p && v != big[u]) {
        head[v] = v; // light child starts its own chain
        decompose(v, u);
    }
}

// O(log n) segtree ranges covering the path u..v (inclusive)
vector<pair<int,int>> getRanges(int u, int v) {
    vector<pair<int,int>> ranges;
    while (true) {
        if (head[u] == head[v]) { // same chain -> final contiguous piece, contains the LCA
            if (lvl[u] > lvl[v]) swap(u, v); // u = higher one = the LCA itself
            ranges.emplace_back(in[u], in[v]);
            // EDGE QUERIES: replace the line above with:
            // if (u != v) ranges.emplace_back(in[u] + 1, in[v]); // skip the LCA slot
            break;
        }
    }
}
```

```

        // jump up from the chain whose head is DEEPER (else we could skip past the LCA)
        if (lvl[head[u]] < lvl[head[v]]) swap(u, v);
        ranges.emplace_back(in[head[u]], in[u]);    // whole chain piece: head..u
        u = par[head[u]];                          // hop above the chain
    }
    return ranges;
}

```

lca.cpp

LCA + K-TH ANCESTOR (binary lifting)

$anc[k][v]$ = the 2^k -th ancestor of v . Build $O(n \log n)$, query $O(\log n)$.

RECALL — how it works:

$anc[0][v]$ = parent. $anc[k][v] = anc[k-1][anc[k-1][v]]$ ("jump $2^{(k-1)}$ twice").

We can fill v 's whole column during the DFS because all of v 's ancestors are already fully computed when we reach v .

kth ancestor: decompose k in binary, jump by each set bit.

LCA(u, v): lift the deeper one to the same level; if equal $\rightarrow$ done;

else jump BOTH up by the largest powers that DON'T make them meet;

they end up just below the LCA $\rightarrow$ answer is $anc[0][u]$.

SETUP (sizes here are placeholders — declare AFTER reading n in real code):

$LOG = \lceil \lg(n) \rceil + 2$; // $2^{LOG} > n$ ($n=2e5 \rightarrow LOG=19$)

$anc.assign(LOG, vector<int>(n));$

$lvl[root] = 0$; $anc[0][root] = root$; // root's "parent" = itself, so jumps CLAMP at root
 $dfs0(root, -1)$;

HANDY: $dist(u, v) = lvl[u] + lvl[v] - 2 * lvl[lca(u, v)]$

kth node on path $u \rightarrow v$: if $k \leq lvl[u] - lvl[l]$ lift from u , else lift $dist - k$ from v .

NOTE: because root points to itself, kth_anc with $k > depth$ just returns the root

(never crashes) — often exactly what you want.

```
int n, LOG;
vector<vector<int>>>anc(LOG, vector<int>(n)), adj(n);
vector<int> lvl(n);

void dfs0(int u, int p) {
    for (auto v : adj[u]) if (v != p) {
        lvl[v] = lvl[u] + 1;
        anc[0][v] = u;

        for (int k=1; k<LOG; k++)
            anc[k][v] = anc[k-1][anc[k-1][v]];    // 2^k = two jumps of 2^(k-1)

        dfs0(v, u);
    }
}

// jump k edges up from u (clamps at root)
int kth_anc(int k, int u) {
    for(int i=0; i<LOG; i++) if(k >> i & 1) u = anc[i][u];
    return u;
}

int get_lca(int u, int v){
    if (lvl[u] > lvl[v]) swap(u, v);    // make v the deeper one
    v = kth_anc(lvl[v]-lvl[u], v);    // equalize levels
    if(u == v) return u;    // u was an ancestor of v

    // biggest-first: jump both up while they stay DIFFERENT
    for(int i=LOG-1; i>=0; i--)
        if(anc[i][u] != anc[i][v])
            u = anc[i][u], v = anc[i][v];

    return anc[0][u];    // now both sit right below the LCA
}
```

mst-kruskal.cpp

MST — KRUSKAL, $O(m \log m)$

Sort edges ascending, greedily take any edge whose endpoints are in different components (DSU). Self-contained mini-DSU included below.

USAGE:

```
Kruskal::init(n); Kruskal::edges.push_back({w, u, v}); ...
ll total = Kruskal::mst(); // -1 if graph is disconnected
Kruskal::treeEdges // the n-1 chosen {w, u, v} (build adj from it)
```

RECALL — properties that solve problems (often no MST code needed at all):

- CUT property: the lightest edge crossing ANY cut is in some MST.
- MST minimizes the MAXIMUM edge over the path for EVERY pair u, v
 -> "minimize the max edge $u \rightarrow v$ " = sort edges + DSU, stop when u, v connect (no tree needed). All-pairs version: path-max on the MST via lca.cpp.
- MAXIMUM spanning tree: sort descending.
- Which edges can be in some MST? Process equal-weight GROUPS together:
 an edge is useless iff its endpoints were already connected BEFORE its group.
- Second-best MST = min over non-tree edges (u, v, w) of $\text{MST} - \text{maxEdgeOnPath}(u, v) + w$.

namespace Kruskal

```
{
    int n; vector<int> par, len;
    vector<array<ll,3>> edges, treeEdges; // {w, u, v}

    void init(int n_) {
        n = n_;
        par.assign(n+3, 0), len.assign(n+3, 1);
        iota(all(par), 0);
        edges.clear(), treeEdges.clear();
    }
    int find(int u) { return par[u] == u ? u : par[u] = find(par[u]); }
    bool merge(int u, int v) {
        u = find(u), v = find(v);
        if (u == v) return false;
        if (len[u] < len[v]) swap(u, v);
        par[v] = u, len[u] += len[v];
        return true;
    }

    ll mst() {
        sort(all(edges)); // by weight (first field)
        ll total = 0;
        for (auto& [w, u, v] : edges)
            if (merge(u, v)) total += w, treeEdges.push_back({w, u, v});
        // n-1 edges <=> connected; anything less means a forest
        return (int)treeEdges.size() == n - 1 ? total : -1;
    }
}
```

scc.cpp

SCC + CONDENSATION (Tarjan, one DFS, $O(n + m)$)

SCC = maximal set of nodes in a DIRECTED graph where everyone reaches everyone.

Condensation = shrink each SCC to one node -> always a DAG.

WHAT THIS FUNCTION DOES (in/out parameters — it REPLACES n and adj!):

```
in : n, adj = original directed graph (0-indexed)
out: n, adj = condensation DAG (multi-edges deduped via `seen`)
    compId[u] = component of original node u (size n_original, pre-sized by caller!)
    sccs[c]   = list of original nodes in component c
Keep copies of the originals if you still need them.
```

CALL:

```
vector<int> compId(n); vector<vector<int>>> sccs;
condense(n, adj, compId, sccs);
```

KEY PROPERTY (Tarjan gives it for free):

```
components are produced in REVERSE topological order ->
for every condensed edge cu -> cv: compId[u] > compId[v].
So looping c = 0..n-1 processes SINKS FIRST — perfect for DAG DP where
dp[c] depends on successors (e.g. "best value reachable from c").
Loop c = n-1..0 for sources-first.
```

RECALL — how Tarjan works:

```
low[u] = lowest discovery time reachable from u using tree edges + back edges
to nodes STILL ON THE STACK (inStack check — cross edges to finished SCCs don't count).
When low[u] == in[u], u is the "root" of an SCC -> pop the stack down to u.
```

CLASSIC USES: 2-SAT, "min nodes to start from to reach all" (= #source comps),

"add min edges to make strongly connected" (= max(#sources, #sinks), 1 comp -> 0).

```
void condense(int& n, vector<vector<int>>& adj, vector<int>& compId, vector<vector<int>>& sccs) {
    vector<bool> inStack(n);
    stack<int> st;
    vector<int> in(n), low(n);    // in = discovery time, 0 = unvisited (timer starts at 1)
    int id = 0, timer = 1;

    function<void(int)> tarjan = [&](int u) {
        in[u] = low[u] = timer++;
        st.push(u); inStack[u] = true;
        for (int v : adj[u]) {
            if (!in[v]) {                // tree edge
                tarjan(v);
                low[u] = min(low[u], low[v]);
            } else if (inStack[v]) {    // back/cross edge into current stack
                low[u] = min(low[u], in[v]);
            }
            // else: v's SCC already finished -> ignore
        }

        if (in[u] == low[u]) {          // u is the root of an SCC -> pop it
            vector<int> comp;
            while (!st.empty()) {
                int x = st.top(); st.pop();
                inStack[x] = false;
                comp.emplace_back(x);
                compId[x] = id;
                if (x == u) break;
            }
            sccs.emplace_back(comp);
            id++;
        }
    };

    for (int i=0; i<n; i++) if (!in[i]) tarjan(i);

    // build condensation, dedup parallel edges between the same pair of comps
    set<pair<int,int>> seen;
    vector<vector<int>> nAdj(id);
    for (int u=0; u<n; u++) {
        for (int v : adj[u]) {
            if (compId[u] != compId[v] && seen.find({compId[u], compId[v]}) == seen.end()) {
                nAdj[compId[u]].emplace_back(compId[v]);
                seen.insert({compId[u], compId[v]});
            }
        }
    }

    n = id;          // n and adj now describe the DAG!
    adj = nAdj;
}
```

topological-sort.cpp

TOPOLOGICAL SORT (Kahn / BFS on in-degrees) — $O(n + m)$

Order of a DAG where every edge $u \rightarrow v$ has u BEFORE v .

USAGE:

```
auto order = toposort(n, adj);
if (sz(order) < n) -> THE GRAPH HAS A CYCLE (that's the standard cycle check!)
```

VARIANTS:

- LEXICOGRAPHICALLY SMALLEST order: replace queue with `priority_queue<int, vector<int>, greater<>>` — $O(n \log n + m)$.
- DP over a DAG (longest path, counting paths): process nodes in this order, push dp along edges. Longest path in DAG = classic "critical path".
- DFS alternative: postorder DFS, reverse it (same as SCC condensation order — note `scc.cpp` already hands you a reverse-topological component order for free).

```
vector<int> toposort(int n, vector<vector<int>>& adj) {
    vector<int> indeg(n);
    for (int u=0; u<n; u++)
        for (int v : adj[u]) indeg[v]++;

    queue<int> q;
    for (int i=0; i<n; i++) if (!indeg[i]) q.push(i);

    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.pb(u);
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push(v);
    }
    return order;
}
```

// size < n <=> cycle exists

Range Queries

fenwick-tree-updatepoint-queryrange.cpp**FENWICK / BIT — point update, range SUM query. $O(\log n)$ each, tiny constant.****!! 1-INDEXED !!** Valid positions are 1..n. Index 0 breaks the loops (i&-i).

USAGE:

```
Fenwick f(n);
f.update(i, x);    // a[i] += x    (to SET a[i]=v: update(i, v - current))
f.query(l, r);     // sum of a[l..r]
```

RECALL — why it works: $t[i]$ stores the sum of the block of length $(i \& -i)$ ending at i . Prefix sum $p(i)$ walks $i \rightarrow i - (i \& -i)$ (strip lowest bit); update walks the other way. Range sum = $p(r) - p(l-1)$.

WHEN Fenwick vs segtree: Fenwick only needs an INVERTIBLE op (sum, xor, count).

min/max range queries -> use segtree/sparse table instead.

XOR variant: replace both `+=` with `^=` and the `-` in query with `^` — done.

CLASSIC TRICKS:

- Count inversions: sweep the array, query how many seen values are $> a[i]$.
- "k-th smallest present value": Fenwick over values + binary lifting descent.
- Values up to $1e9$? coordinate-compress first (Other/coordinate-compression.cpp).

Range UPDATE + point query -> see the sibling file (difference-array Fenwick).

Range update + range query -> needs 2 Fenwicks or a lazy segtree.

```
class Fenwick {
    int n;
    vector<ll> t;

    ll p(int i) { // prefix sum a[1..i]
        ll ret = 0;
        for (; i>0; i -= i&-i) ret += t[i];
        return ret;
    }

public:
    Fenwick(int _n):n(_n) { t.assign(n+1,0LL); }

    ll query(int l, int r) { return p(r) - p(l-1); }

    void update(int i, ll x) { // add x to a[i]
        if (i < 1) return; // guard: i=0 has i&-i == 0 -> would loop forever
        for (; i<=n; i += i&-i) t[i] += x;
    }
};
```

fenwick-tree-updaterange-querypoint.cpp**FENWICK / BIT — RANGE update, POINT query (difference-array trick). $O(\log n)$.****!! 1-INDEXED !!** Valid positions are 1..n.

USAGE:

```
Fenwick f(n);
f.update(l, r, x);    // a[i] += x for ALL i in [l, r]
f.query(i);           // current value of a[i]
```

RECALL — the trick: keep Fenwick over the DIFFERENCE array d ($d[i] = a[i] - a[i-1]$).

```
"add x on [l..r]" = d[l] += x, d[r+1] -= x    (two point updates)
"value of a[i]"   = prefix sum d[1..i]        (one prefix query)
```

The array starts as all zeros. If you have initial values, either:

- do `update(i, i, a0[i])` for each i, or
- just add `a0[i]` yourself when reading `query(i)`.

```
class Fenwick {
    int n;
    vector<ll> t;
    void u(int i, ll x) { // point-update the underlying diff-array Fenwick
        for (; i <= n; i += i & -i) t[i] += x;
    }
public:
    Fenwick(int _n):n(_n) { t.assign(n+1,0LL); }
    ll query(int i) { // returns a[i] = prefix sum of diffs
        ll ret = 0;
        for (; i > 0; i -= i & -i) ret += t[i];
        return ret;
    }
    void update(int l, int r, ll x) { // add x to a[l..r]
        if (l > r || r < 1) return;
        if (l < 1) l = 1;           // clamp instead of crashing
        u(l, x);
        if (r+1 <= n) u(r+1, -x);   // r == n -> no cancel needed
    }
};
```

lazy-seg-tree.cpp**LAZY SEGMENT TREE — RANGE update, range query. $O(\log n)$ each.**

0-INDEXED, query/update(l, r) INCLUSIVE. As written: RANGE ADD + RANGE SUM.

USAGE:

```
SegmentTree st(a);           // builds from vector a in O(n)
st.update(l, r, x);          // a[i] += x for all i in [l, r]
st.query(l, r);              // sum a[l..r]
```

RECALL — lazy propagation:

A node's val is always CORRECT for its whole segment; pending updates that its CHILDREN haven't seen yet sit in `lazy`. Before stepping into children (query or update), propagate() pushes the lazy one level down.

change(x, len) = "apply update x to a whole segment of length len".

===== THE 3 SPOTS THAT DEFINE THE TREE (change these per problem) =====

```
1) merge()           - how two children combine      (here: sum)
2) Node neutral      - identity of merge              (sum->0, min->-INF, max->-INF)
3) change()          - how an update hits a segment + how lazies COMPOSE
```

===== COMMON VARIANTS =====

(A) range ADD + range SUM (as written):

change: val += x * len; lazy += x;

(B) range ADD + range MIN/MAX:

merge -> min/max, neutral -> INF/-INF,

change: val += x; lazy += x; // NO len - min shifts by x, not x*len

(C) range ASSIGN (a[i] = x) + range SUM:

change: val = x * len; lazy = x; // lazies OVERWRITE instead of add

and DEFAULT_LAZY must be a value that can never be assigned (isLazy already guards this - that's exactly why isLazy exists: 0 may be a legit assignment!)

(D) assign + min/max: change: val = x; lazy = x;

PITFALLS:

- val/lazy are ll here: range-add + sum overflows int almost immediately.
- neutral Node() is returned for out-of-range parts of query - it MUST match merge.
- Multiple lazy types (add AND assign together) need composition rules - derive carefully (assign kills pending add; add on top of assign folds into the assign).

```
struct Node {
    static ll DEFAULT_VALUE, DEFAULT_LAZY;
    ll val, lazy;
    bool islazy;

    Node(ll x = DEFAULT_VALUE) : val(x), lazy(DEFAULT_LAZY), isLazy(false) {}

    // apply update x to this ENTIRE segment (len = number of leaves under it)
    void change(ll x, int len) {
        val += x * len;          // (A) sum. | (B) min/max: val += x | (C) assign+sum: val = x*len
        lazy += x;               // compose with pending lazy.      | (C)/(D) assign: lazy = x
        isLazy = true;
    }
};
ll Node::DEFAULT_VALUE = 0, Node::DEFAULT_LAZY = 0; // neutral val / neutral lazy

class SegmentTree {
private:
    int TREESIZE;
    vector<Node> tree;

    Node merge(const Node &n1, const Node &nr) { return Node(n1.val + nr.val); } // CHANGE

    void Build(const vector<ll> &a, int idx, int lx, int rx) {
        if (lx == rx) {
            if (lx < (int) a.size()) tree[idx] = Node(a[lx]);
            else tree[idx] = Node(); // padding leaves = neutral
            return;
        }

        int mid = (lx + rx) >> 1;
        Build(a, 2 * idx + 1, lx, mid);
        Build(a, 2 * idx + 2, mid + 1, rx);

        tree[idx] = merge(tree[2 * idx + 1], tree[2 * idx + 2]);
    }

    // push this node's pending lazy down ONE level (children lengths differ!)
    void propagate(int idx, int lx, int rx) {
        if (lx == rx || !tree[idx].isLazy) return;

```

```

    int mid = (lx + rx) >> 1;
    tree[2 * idx + 1].change(tree[idx].lazy, mid - lx + 1);
    tree[2 * idx + 2].change(tree[idx].lazy, rx - mid);

    tree[idx].isLazy = false;
    tree[idx].lazy = Node::DEFAULT_LAZY;
}

Node query(int l, int r, int idx, int lx, int rx) {
    propagate(idx, lx, rx); // ALWAYS push before descending

    if (lx >= l && rx <= r) return tree[idx]; // fully inside
    if (rx < l || lx > r) return Node(); // fully outside -> neutral

    int mid = (lx + rx) >> 1;
    Node nLeft = query(l, r, 2 * idx + 1, lx, mid);
    Node nRight = query(l, r, 2 * idx + 2, mid + 1, rx);

    return merge(nLeft, nRight);
}

void update(int l, int r, ll newVal, int idx, int lx, int rx) {
    propagate(idx, lx, rx);

    if (lx >= l && rx <= r) { // fully inside -> lazy-apply and stop
        tree[idx].change(newVal, rx - lx + 1);
        return;
    }
    if (rx < l || lx > r) return;

    int mid = (lx + rx) >> 1;
    update(l, r, newVal, 2 * idx + 1, lx, mid);
    update(l, r, newVal, 2 * idx + 2, mid + 1, rx);

    tree[idx] = merge(tree[2 * idx + 1], tree[2 * idx + 2]);
}

public:
    SegmentTree(const vector<ll> &a) {
        TREESIZE = 1;
        while (TREESIZE < (int) a.size()) TREESIZE <= 1; // pad to power of two
        tree.resize(2 * TREESIZE);
        Build(a, 0, 0, TREESIZE - 1);
    }

    ll query(int l, int r) { return query(l, r, 0, 0, TREESIZE - 1).val; }

    void update(int l, int r, ll newVal) { update(l, r, newVal, 0, 0, TREESIZE - 1); }
};

```

mo.cpp

MO'S ALGORITHM — OFFLINE range queries in $O((n + q) * \sqrt{n} * \text{cost}(\text{add}/\text{del}))$

Use when: no updates, all queries known upfront, and the answer for $[l, r]$

can be MAINTAINED while moving the endpoints one step at a time.

Classics: # distinct values in range, # pairs, mode-ish counts, XXX-frequency stuff.

RECALL — why it's fast:

Sort queries by (block of l , then r). Inside one block R only sweeps $\sim n$ total (amortized), and L jiggles within a block of size SQ per query.

The $\lceil l/SQ \rceil \& 1 ? r > \dots : r < \dots$ is the SNAKE/parity order: alternate R 's sweep direction per block $\rightarrow R$ doesn't reset to the left each block ($\sim 2x$ speedup).

SETUP — needs `const int SQ` defined:

$SQ \sim \sqrt{n}$ (450 for $n=2e5$), or tuned: $SQ = \max(1, n / \max(1, (\text{int})\sqrt{q}))$.

!! INDEXING: this template starts $L=1, R=0$ (empty window) $\rightarrow$ queries are

1-INDEXED inclusive. Keep your array 1-indexed to match.

FILL IN (the whole problem lives in these three lambdas):

`add(i)` — element $a[i]$ enters the window (e.g. if $(++\text{cnt}[a[i]] == 1)$ `distinct++`;))

`del(i)` — element $a[i]$ leaves the window (e.g. if $(--\text{cnt}[a[i]] == 0)$ `distinct--`;))

`answer()` — current window's answer

Values up to $1e9 \rightarrow$ coordinate-compress first so `cnt[]` can be an array.

ORDER MATTERS below: expand (add) BEFORE shrink (del) so the window never goes "inside out" ($L > R+1$) — with e.g. `cnt[]--` on absent elements that corrupts state.

If delete is impossible/hard (max-so-far, DSU...) $\rightarrow$ `mo-with-rollback.cpp`.

Path queries on a tree $\rightarrow$ `mo-on-tree.cpp`.

```
struct Query
{
    int l, r, idx; // idx = original position (answers go back in order)
    bool operator<(const Query& other) const {
        if (l/SQ != other.l/SQ) return l/SQ < other.l/SQ; // primary: block of l
        return ((l/SQ) & 1) ? r > other.r : r < other.r; // snake order on r
    }
};

void MO(vector<Query>& queries, vector<int>& ans)
{
    sort(queries.begin(), queries.end());

    auto add = [&](int i) {
        // a[i] enters the window
    };

    auto del = [&](int i) {
        // a[i] leaves the window
    };

    auto answer = [&]() {
        return 0; // current window answer
    };

    int L = 1, R = 0; // empty window
    for (auto [l, r, idx] : queries) {
        while (l < L) add(--L); // expand left
        while (R < r) add(++R); // expand right
        while (L < l) del(L++); // shrink left
        while (r < R) del(R--); // shrink right

        ans[idx] = answer();
    }
}
```

mo-on-tree.cpp

MO'S ON A TREE — offline PATH queries $u..v$, $O((n+q) * \sqrt{n})$ Classic: SPOJ COT2 — # distinct values on the path. Needs: $\text{const SQ} (\sim \sqrt{2n})$, LOG.

RECALL — the flattening trick (Euler tour with BOTH stamps):

Every node u is written TWICE: at $\text{in}[u]$ (entering) and $\text{out}[u]$ (leaving). $\text{flat}[]$ has length $2n$. For a path query $u..v$ (say $\text{in}[u] \leq \text{in}[v]$):

- u is an ancestor of v ($\text{lca} == u$): $\text{window} = [\text{in}[u], \text{in}[v]]$
- otherwise: $\text{window} = [\text{out}[u], \text{in}[v]]$ + add LCA manually

KEY FACT: inside that window, nodes ON the path appear exactly ONCE;

nodes OFF the path appear TWICE (their whole subtree got entered and left).

So $\text{process}()$ toggles: 1st appearance $\rightarrow$ node enters, 2nd $\rightarrow$ node leaves.That's the freqNode parity check. The LCA is the only path node missing in

case 2, hence the manual add/del around answering.

PITFALLS:

- $\text{a}[i]$ values used as indices into cnt arrays $\rightarrow$ COORDINATE-COMPRESS them if they can exceed n (here input is just shifted to 0-based: values assumed $1..n$; replace with compression when values go to $1e9$).
- add/del receive a POSITION in $\text{flat}[]$; $x = \text{flat}[i]$ is the VALUE at that spot.
- Don't forget to define SQ and LOG ($\text{LOG} = _lg(n)+2$).

```

struct Query {
    int l, r, iq, lca;                // window [l,r] in flat[], iq = original index,
                                     // lca = -1 if ancestor case (LCA already inside window)
    bool operator<(const Query& other) const {
        if (l/SQ != other.l/SQ) return l/SQ < other.l/SQ;
        return ((l/SQ)&1) ? r > other.r : r < other.r;    // snake order (see mo.cpp)
    }
};

void solve()
{
    int n, q; cin >> n >> q;
    vector<int> a(n);
    for (int i=0; i<n; i++) cin >> a[i], --a[i];    // values -> 0-based (COMPRESS if large!)
    vector<vector<int>> adj(n);
    for (int i=0; i<n-1; i++) {
        int u, v; cin >> u >> v;
        adj[--u].pb(--v);
        adj[v].pb(u);
    }

    // ---- Euler tour (in & out stamps) + binary lifting for LCA ----
    int timer = 0;
    vector<int> in(n), out(n), lvl(n), flat(2*n), node(2*n); // node[pos] = which node sits there
    vector<vector<int>> anc(LOG, vector<int>(n));
    function<void(int,int)> dfs0 = [&](int u, int p) {
        in[u] = timer;
        node[timer++] = u;
        for (int v : adj[u]) if (v != p) {
            lvl[v] = lvl[u] + 1;

            anc[0][v] = u;
            for (int i=1; i<LOG; i++)
                anc[i][v] = anc[i-1][anc[i-1][v]];

            dfs0(v, u);
        }

        out[u] = timer;
        node[timer++] = u;
    };
    dfs0(0, -1);

    for (int i=0; i<n; i++) flat[in[i]] = flat[out[i]] = a[i]; // value copied to both stamps

    auto kth_ancestor = [&](int k, int u) {
        for (int i=0; i<LOG; i++) if (k>>i&1) u = anc[i][u];
        return u;
    };

    auto get_lca = [&](int u, int v) {
        if (lvl[u] > lvl[v]) swap(u, v);
        v = kth_ancestor(lvl[v]-lvl[u], v);
        if (u == v) return u;
        for (int i=LOG-1; i>=0; i--)
            if (anc[i][u] != anc[i][v])
                u = anc[i][u], v = anc[i][v];
        return anc[0][u];
    };

```

```

};

// ---- build the windows (the case split from the header) ----
vector<Query> queries(q);
for (int i=0; i<q; i++) {
    int u, v; cin >> u >> v, --u, --v;
    if (in[u] > in[v]) swap(u, v);          // ensure in[u] <= in[v]
    int lca = get_lca(u, v);
    if (lca == u) {
        queries[i] = {in[u], in[v], i, -1}; // ancestor case: LCA(=u) is in the window
    }
    else {
        queries[i] = {out[u], in[v], i, lca}; // general case: add LCA by hand later
    }
}
sort(all(queries));

vector<int> answer(q), freqNode(n);          // freqNode = appearances of node in window (0/1/2)

// FILL THESE (same roles as plain Mo): maintain answer over the SET of path nodes
auto add = [&](int i) {
    int x = flat[i];
    // x = the VALUE entering, e.g.: if (++cnt[x] == 1) distinct++;
};

auto del = [&](int i) {
    int x = flat[i];
    // x = the VALUE leaving, e.g.: if (--cnt[x] == 0) distinct--;
};

// parity toggle: 1st time node appears -> enters; 2nd time -> cancels out
auto process = [&](int i) {
    int u = node[i];
    if (++freqNode[u] & 1) add(in[u]);
    else del(in[u]);
};

auto get_answer = [&] {
    return 0; // current answer, e.g. distinct
};

int L = 0, R = -1;                          // 0-indexed empty window (flat is 0-indexed!)
for (const auto& [l, r, iq, lca] : queries) {
    while (l < L) process(--L);
    while (R < r) process(++R);
    while (L < l) process(L++);
    while (r < R) process(R--);
    if (~lca) add(in[lca]);                  // LCA joins just for the answer...
    answer[iq] = get_answer();
    if (~lca) del(in[lca]);                  // ...and leaves again (keeps window clean)
}

for (int x : answer) cout << x << '\n';
}

```

mo-with-rollback.cpp**MO WITH ROLLBACKS ("Mo, add-only") — SKELETON, fill the marked spots**

Use when ADD is easy but DELETE is impossible/ugly:

running max / "max frequency" / merging DSU components / max subarray-ish info.

Complexity: $O(n \cdot \sqrt{q} + q \cdot \sqrt{n})$ adds — no deletes ever happen.

RECALL — how deletes are avoided:

Group queries by BLOCK OF L; inside a block sort by r ASCENDING.

For one block [lBlock, rBlock]:

- R starts at rBlock+1 and only GROWS through the block's queries
-> right-side adds are permanent for the block, never undone.
- The left part (l .. rBlock) is small ($\leq \sqrt{q}$ elements): add it, answer, then ROLLBACK just those adds (undo via a stack of old values).
- Tiny queries fully inside the block ($r \leq rBlock$) -> brute force over $\leq \sqrt{q}$ elements directly, then undo.

Rollback = restore recorded old values in reverse order. NO delete logic needed — that's the whole point (e.g. you can't "un-max" a running maximum, but you CAN restore its previous snapshot).

TO USE: paste into solve(), declare n, q, SQ ($\sim \sqrt{n}$), the array, and `vector<int> answer(q);` then fill every `//` hole below.

```
class MO_With_Rollbacks {
    struct Query {
        int l, r, iq;
        bool operator<(const Query& other) const {
            return r < other.r;           // r ascending WITHIN a block (no snake here!)
        }
    };

    struct State {
        // one undo record: whatever add() overwrites, e.g. {int* where, int oldVal}
        // or {value, oldCnt} — enough to restore EXACTLY one add()'s damage
    };

    MO_With_Rollbacks(vector<int>& a, vector<pair<int,int>>& quer) {}

    vector<int> MO()
    {
        int bCnt = (n+SQ-1)/SQ;           // number of 1-blocks
        vector<vector<Query>> queries(bCnt); // bucket queries by block of 1
        for (int i=0; i<q; i++) {
            int l, r; cin >> l >> r, --l, --r;
            queries[l/SQ].pb({l,r,i});
        }
        for (auto& qu : queries) sort(all(qu));

        int checkpoint, ans;               // ans = CURRENT maintained answer
        stack<State> changes;               // undo log

        auto reset = [&] {
            // wipe the whole structure (called once per block): cnt[]=0, ans=0, clear changes
        };

        auto rollback = [&] {
            // undo everything after `checkpoint`
            while (sz(changes) > checkpoint) {
                auto& c = changes.top(); changes.pop();
                // restore c's old values (reverse order is automatic via the stack)
            }
        };

        auto add = [&] (int idx) {
            // a[idx] enters: PUSH the old state onto `changes` FIRST, then apply.
            // NOTE: `ans` itself must also be restorable -> record it in State too,
            // or recompute it; easiest is to save {oldAns, ...} in every record.
        };

        for (int ib=0; ib<bCnt; ib++) {
            reset();                       // fresh structure for this block

            int lBlock = ib*SQ, rBlock = min(n-1, lBlock+SQ-1);
            int R = rBlock + 1;             // right pointer, only moves forward

            for (const auto& [l, r, iq] : queries[ib]) {
                if (r <= rBlock) {          // tiny query, lives inside the block
```

```
        // brute force: for (int j=l; j<=r; j++) add(j);

        answer[iq] = ans;

        // rollback(); (undo ALL of it – set checkpoint=0 before, or loop-pop)
        continue;
    }

    while (R <= r) add(R++);    // permanent right-side adds (survive the block)
    checkpoint = sz(changes);  // <- everything after this will be undone

    for (int j=rBlock; j>=1; j--) add(j);    // temporary left part

    answer[iq] = ans;

    rollback();                // undo only the left part
}
}

return answer;
};
```

seg-tree.cpp

SEGMENT TREE (recursive) — point update, range query. $O(\log n)$ each.

0-INDEXED. query(l, r) is INCLUSIVE on both ends.

USAGE:

```
SegmentTree st; st.init(n);
st.update(pos, val);    // a[pos] = val (assignment, not +=; change in update() if needed)
st.query(l, r);
```

CHANGE HERE (the only two spots that define "what the tree computes"):

```
1) merge()    - sum / min / max / gcd / max-subarray-struct / ...
2) Node()     - the NEUTRAL element, must satisfy merge(x, neutral) = x:
                  sum -> 0, min -> INF, max -> -INF, gcd -> 0.
                (query returns Node() for fully-outside ranges, so a wrong neutral = wrong answers!)
Put anything you need inside Node (e.g. {sum, prefix, suffix, best} for max subarray)
and combine the fields in merge().
```

NOTES:

- Size padded up to a power of two; leaves beyond n hold Node() so they're harmless.
- Memory: allocates $4 \cdot \text{TREESIZE}$ but $2 \cdot \text{TREESIZE}$ is enough (TREESIZE already padded) – wasteful but safe; shrink if memory-tight.
- To BUILD from an initial array in $O(n)$: see lazy-seg-tree.cpp's Build (same idea), or just call update n times ($O(n \log n)$, usually fine).

```
class SegmentTree {
    struct Node {
        int val;
        Node(int x = 0) : val(x) {}    // CHANGE: neutral element (0 for sum, INF for min...)
    };

    int TREESIZE;
    vector<Node> tree;

    Node merge(const Node& nl, const Node& nr) { return Node(nl.val + nr.val); } // CHANGE

    Node query(int l, int r, int idx, int lx, int rx) {
        if (rx < l || lx > r) return Node();    // fully outside -> neutral
        if (lx >= l && rx <= r) return tree[idx]; // fully inside -> take node

        int mid = (lx + rx) >> 1;    // children: 2i+1 = [lx,mid], 2i+2 = (mid,rx]
        return merge(query(l, r, 2*idx+1, lx, mid), query(l, r, 2*idx+2, mid+1, rx));
    }

    void update(int pos, int newVal, int idx, int lx, int rx) {
        if (lx == rx) {    // reached the leaf
            tree[idx] = Node(newVal);    // CHANGE here for a[pos] += x style updates
            return;
        }

        int mid = (lx + rx) >> 1;
        if (pos <= mid) update(pos, newVal, 2*idx+1, lx, mid);
        else update(pos, newVal, 2*idx+2, mid+1, rx);

        tree[idx] = merge(tree[2*idx+1], tree[2*idx+2]);    // recompute on the way up
    }

public:
    SegmentTree(){}

    void init(const int& n) {
        TREESIZE = 1;
        while (TREESIZE < n) TREESIZE <= 1;    // pad to power of two
        tree.assign(4 * TREESIZE, Node());    // 2*TREESIZE suffices – see header
    }

    int query(int l, int r) { return query(l, r, 0, 0, TREESIZE-1).val; }

    void update(int pos, int newVal) { update(pos, newVal, 0, 0, TREESIZE-1); }
};
```

seg-tree-iterative.cpp**SEGMENT TREE (iterative / "bottom-up") — point update, range query**

Same power as the recursive one but ~2-3x faster constant and works with ANY n (no power-of-two padding; memory exactly 2n).

0-INDEXED, query(l, r) INCLUSIVE (converted to half-open [l, r+1) internally).

LAYOUT RECALL: leaves live at tree[n .. 2n-1] (leaf i at n+i),
internal node i has children 2i and 2i+1, parent of i is i/2.

Query climbs from both ends inward, grabbing a node whenever it's
"sticking out" of the range (the l&1 / r&1 checks).

CHANGE HERE:

- 1) merge() — sum / min / max / gcd / ...
- 2) Node() — the NEUTRAL element matching merge (sum->0, min->INF, ...)
- 3) update() — currently ASSIGNS; make it `tree[pos].val += val` for increments.

!! CAVEAT — NON-COMMUTATIVE merges (matrices, "function composition", hashes):
this query accumulates the right side in reverse order. Fine for sum/min/max;
for non-commutative ops keep two accumulators (resL, resR) and merge
resL = merge(resL, tree[l++]) / resR = merge(tree[--r], resR), answer merge(resL, resR).

BUILD from array in O(n): fill tree[n+i] = Node(a[i]),
then for (i = n-1; i >= 1; i--) tree[i] = merge(tree[2i], tree[2i+1]).

```
class SegmentTree {
    struct Node {
        int val;
        Node(int x = 0) : val(x) {}      // CHANGE: neutral element
    };

    int n;
    vector<Node> tree;

    Node merge(const Node& nl, const Node& nr) { return Node(nl.val + nr.val); } // CHANGE

    Node queryy(int l, int r) {          // half-open [l, r)
        Node res = Node();
        for (l+=n, r+=n; l < r; l>>=1, r>>=1) {
            if (l&1) res = merge(res, tree[l++]); // l is a right child -> take it, move right
            if (r&1) res = merge(res, tree[--r]);  // r is a right child -> take the one before
        }
        return res;
    }

public:
    SegmentTree(const int& _n){ init(_n); }

    void init(const int& _n) {
        n = _n;
        tree.assign(n<<1, Node()); // exactly 2n nodes
    }

    void update(int pos, int val) {      // a[pos] = val
        pos += n;                        // jump to the leaf
        tree[pos] = Node(val);
        while (pos > 1) {                 // fix ancestors up to the root
            pos >>= 1;
            tree[pos] = merge(tree[pos << 1], tree[pos << 1 | 1]);
        }
    }

    int query(int l, int r) { return queryy(l, r+1).val; } // inclusive wrapper
};
```

sliding-window-minmax.cpp**SLIDING WINDOW MIN/MAX — monotonic deque, $O(n)$ total**

Min of every window of size k . Deque holds INDICES with $a[.]$ strictly increasing front→back, so the front is always the window minimum.

An element killed from the back can never be an answer again (something newer AND smaller-or-equal exists) — that's the whole proof.

USAGE: `windowMin(a, k) -> res[i] = min of  $a[i .. i+k-1]$  (size  $n-k+1$ ).`

For max: flip the comparison (or negate the array).

RECALL:

- DP speedup: $dp[i] = cost[i] + \min(dp[i-k .. i-1])$ -> keep the deque over dp instead of re-querying a segment tree: $O(n)$ instead of $O(n \log n)$.
- Variable-width windows (two pointers): same deque; pop front while its index < current left bound.
- Monotonic STACK cousin — nearest smaller to the left for every i :
`while (!st.empty() && a[st.top()] >= a[i]) st.pop();` // top = answer, then push i
 Gives "span until a smaller element" -> largest rectangle in histogram.

```
vector<ll> windowMin(vector<ll>& a, int k)
{
    int n = a.size();
    vector<ll> res;
    deque<int> dq; // indices, a[dq] increasing
    for (int i = 0; i < n; i++) {
        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back(); // > for max
        dq.push_back(i);
        if (dq.front() <= i - k) dq.pop_front(); // fell out of window
        if (i >= k - 1) res.push_back(a[dq.front()]);
    }
    return res;
}
```

sparse-table.cpp**SPARSE TABLE — STATIC array, $O(n \log n)$ build, $O(1)$ range query**

No updates allowed! If you need updates -> segment tree.

USAGE: SparseTable st(a); st.query(l, r); // 0-indexed, inclusive

RECALL — how it works:

$t[k][i]$ = merge of the 2^k elements starting at i .

Query $[l..r]$: take the two 2^k blocks covering the range from each end — they OVERLAP in the middle, which is why the op must be IDEMPOTENT:

>>> merge must satisfy $f(x, x) = x$: min / max / gcd / AND / OR — YES

>>> sum / xor / count — NO! (overlap counts twice)

for sums use prefix sums; or query the disjoint $O(\log n)$ way.

CHANGE HERE: `merge` (currently min).

$\text{__lg}(x) = \text{floor}(\log_2(x))$ (GCC builtin). Build memory: $O(n \log n)$ ints.

```
class SparseTable {
private:
    vector<vector<int>> t;
    int n, LOG;
    int merge(int e, int o) { return min(e, o); } // CHANGE: min/max/gcd/AND/OR only!

public:
    SparseTable(vector<int>& a) {
        n = a.size(), LOG = 32 - __builtin_clz(n); // = floor(log2(n)) + 1
        t.resize(LOG, vector<int>(n));

        for (int i=0; i<n; i++) t[0][i] = a[i]; // level 0 = the array itself

        for (int k=1; (1<<k)<=n; k++) {
            for (int i=0; i+(1<<k)<=n; i++) {
                // block of  $2^k$  = two adjacent blocks of  $2^{(k-1)}$ 
                t[k][i] = merge(t[k-1][i], t[k-1][i+(1<<(k-1))]);
            }
        }

        int query(int l, int r) { // 0-indexed inclusive, needs  $l \leq r$ 
            int k = __lg(r-l+1);
            return merge(t[k][l], t[k][r-(1<<k)+1]); // two overlapping blocks
        }
};
```

sqrt-decomposition.cpp**SQRT DECOMPOSITION — $O(\sqrt{n})$ per query/update, dead simple to bend**

Split the array into blocks of size $SQ \sim \sqrt{n}$; keep an aggregate per block.

Query $[l, r]$: whole blocks inside use the aggregate, the two ragged ends brute force.

WHEN over a segtree (which is $O(\log n)$ and usually better for plain sum/min):

- the block aggregate is something a segtree can't merge easily:
SORTED block (-> "count elements > x in range" via binary search per block),
frequency map per block, "best pair inside block", etc.
- lazy-style tricks: tag whole blocks, rebuild only ragged blocks (rebuild = $O(SQ)$).
- you're out of time to debug a segtree – this is 15 lines and hard to get wrong.

As written: point ASSIGN + range SUM, 0-indexed inclusive.

CHANGE HERE: what `b[]` stores + how update/query maintain it (see WHEN above).

```
int SQ;
vector<ll> a, b;                                // b[k] = aggregate (sum) of block k

void build(int n) {                             // call after filling a; O(n)
    SQ = (int)sqrtl(n) + 1;
    b.assign(n / SQ + 1, 0);
    for (int i=0; i<n; i++) b[i / SQ] += a[i];
}

void update(int i, ll x) {                      // a[i] = x
    b[i / SQ] += x - a[i];                     // fix the aggregate, not rebuild
    a[i] = x;
}

ll query(int l, int r) {                       // sum of a[l..r]
    ll sum = 0;
    for (int i=l; i<=r; ) {
        if (i % SQ == 0 && i + SQ - 1 <= r) { // block fully inside -> take aggregate
            sum += b[i / SQ];
            i += SQ;
        }
        else sum += a[i++];                   // ragged edge -> element by element
    }
    return sum;
}
```

Strings

binary-trie.cpp

BINARY TRIE — numbers as bit-strings (MSB first). THE tool for XOR problems.

insert / del / maxXor in $O(\text{MAXBITS})$ each.

CHANGE HERE: MAXBITS = highest bit index.

29 covers values $< 2^{30}$. For long long values change: `int -> ll`, `1<<i -> 1LL<<i`,
`MAXBITS -> 59 (or 62)`, and ``bool c = n>>i&1`` stays fine.

RECALL — why a trie: walking from the top bit, at each level you can CHOOSE the child that makes the current answer-bit 1 (greedy is optimal because a higher bit beats all lower bits combined).

`freq[c]` = how many stored numbers go through child `c` (supports duplicates;
`del()` frees a branch when its `freq` hits 0, so ``child != NULL` == "branch alive"`).

CLASSIC USES:

- max XOR pair in an array: insert all, `maxXor(a[i])` over `i`
- max XOR subarray: prefix XORs + this
- online "insert numbers / query best xor with `x`" sets

PITFALL: `del(n)` assumes `n` IS in the trie (like `trie.cpp`). `maxXor` assumes the trie is non-empty.

```
class BinaryTrie {
    static const int MAXBITS = 29;          // CHANGE: 29 -> values < 2^30; 59+11 for big values
    class Node {
    public:
        Node* child[2];
        int freq[2];                        // freq[c] = numbers passing through child c
        Node() {
            memset(child,0,sizeof child);
            memset(freq,0,sizeof freq);
        }
    };
    Node* root = new Node();

public:
    void insert(int n) {
        Node* cur = root;
        for (int i=MAXBITS; i>=0; i--) {
            bool c = n>>i&1;
            if (cur->child[c] == NULL) cur->child[c] = new Node();
            cur->freq[c]++;
            cur = cur->child[c];
        }
    }

    // unwind the path, decrement freqs, free dead branches
    void del(Node* cur, int n, int i) {
        if (i == -1) return;
        bool c = n>>i&1;
        del(cur->child[c], n, i-1);

        cur->freq[c]--;

        if (!cur->freq[c]) {
            delete cur->child[c];
            cur->child[c] = NULL;
        }
    }

    void del(int n) {                          // !! only call if n is currently in the trie
        del(root,n,MAXBITS);
    }

    // max of (x XOR y) over all stored y — greedy: prefer the OPPOSITE bit
    int maxXor(int x) {
        Node* cur = root;
        int res = 0;
        for (int i=MAXBITS; i>=0; i--) {
            bool c = x>>i&1;
            if (cur->child[!c]) {                // opposite bit available -> answer bit = 1
                res |= 1<<i;                    // (1LL<<i for the ll version!)
                cur = cur->child[!c];
            }
            else cur = cur->child[c];            // forced to match -> answer bit = 0
        }
        return res;
    }
};
```

hashing.cpp

STRING HASHING (polynomial, DOUBLE hash) — substring compare in O(1)

hash($s[0..i]$) = $s[0]*b^i + s[1]*b^{i-1} + \dots + s[i] \pmod{m}$, for TWO independent (base, mod) pairs $\rightarrow$ collision chance $\sim 1/1e18$, safe to treat "hashes equal" as "strings equal".

USAGE:

```
init(); // ONCE globally (fills power tables, needs N >= max length)
Hash h(s); // O(|s|)
h.get(l, r); // pair hash of s[l..r], 0-INDEXED INCLUSIVE, O(1)
Hash h2(t); h.get(..) == h2.get(..) // cross-string compare works (same bases)
```

RECALL — how get() works: $h1[r]$ holds hash of prefix $0..r$; subtract prefix $0..l-1$ SHIFTED UP by $b^{(r-l+1)}$ to align, i.e. $hash(l..r) = h[r] - h[l-1]*b^{len}$.

NOTES / PITFALLS:

- chars are mapped to (value + 1) so no character is 0 — otherwise "a" and "aa" would collide (leading zeros vanish).
- $b1, b2$ must be $>$ alphabet size. For mixed case/digits use e.g. 131, 137.
- vs anti-hash hacks (Codeforces open hacking): randomize the bases at runtime, e.g. $b1 = rand(256, mod1-2)$ using Other/random.cpp.
For ECPC onsite nobody hacks you — fixed bases are fine.

CLASSIC USES:

- compare any two substrings / check palindrome (hash the reversed string too)
- longest common prefix of two suffixes via binary search + get()
- count distinct substrings of each length, string matching without KMP

```
const int b1 = 41, b2 = 37, mod1 = 1e9+7, mod2 = 1e9+9;
int pw1[N], pw2[N]; // pw[i] = b^i (precomputed powers)

void init() {
    pw1[0] = pw2[0] = 1;
    for (int i=1; i<N; i++) {
        pw1[i] = 1LL * pw1[i-1] * b1 % mod1;
        pw2[i] = 1LL * pw2[i-1] * b2 % mod2;
    }
}

class Hash
{
    int n;
    vector<int> h1, h2; // prefix hashes

public:
    Hash(const string& s): n(s.size()), h1(n), h2(n) {
        h1[0] = h2[0] = (unsigned char)s[0] + 1; // +1: never map a char to 0!
        for (int i=1; i<n; i++) {
            int d = (unsigned char)s[i] + 1;
            h1[i] = (1LL*h1[i-1]*b1%mod1 + d)%mod1;
            h2[i] = (1LL*h2[i-1]*b2%mod2 + d)%mod2;
        }
    }

    pair<int,int> get(int l, int r) { // hash of s[l..r], 0-indexed inclusive
        int x = (h1[r] - (1LL*(l?h1[l-1]:0)*pw1[r-l+1]%mod1) + mod1) % mod1;
        int y = (h2[r] - (1LL*(l?h2[l-1]:0)*pw2[r-l+1]%mod2) + mod2) % mod2;
        return {x, y};
    }
};
```

kmp.cpp**PREFIX FUNCTION + KMP MATCHING — $O(n)$**

$\text{pi}[i]$ = length of the longest PROPER prefix of $s[0..i]$ that is also a suffix
 of $s[0..i]$ ("border" length). e.g. "abcab" $\rightarrow$ $\text{pi} = 0\ 0\ 0\ 1\ 2$

RECALL — the loop: to extend position i , try the previous border $j = \text{pi}[i-1]$;
 if $s[i]$ doesn't match $s[j]$, FALL BACK to the border-of-the-border $\text{pi}[j-1]$,
 repeat. Borders of borders enumerate ALL borders — that's the whole trick.

PATTERN MATCHING: build pi on $p + \# + t$ ($\#$ = any char not in either
 string — it stops borders from crossing the boundary). $\text{pi} == |p| \rightarrow$ full match.

CLASSIC FACTS (asked constantly):

- smallest PERIOD of $s = n - \text{pi}[n-1]$; it tiles s perfectly iff $n \% \text{period} == 0$.
- all borders of s : $\text{pi}[n-1]$, $\text{pi}[\text{pi}[n-1]-1]$, ... (chain down to 0).
- count occurrences of every prefix / build prefix automaton — extensions of pi .
- "shortest string with s as prefix and suffix"-type tricks: think borders.

```
vector<int> prefixFunction(const string& s) {
    int n = sz(s);
    vector<int> pi(n);
    for (int i=1; i<n; i++) {
        int j = pi[i-1];
        while (j && s[i] != s[j]) j = pi[j-1]; // best border so far
        if (s[i] == s[j]) j++; // fall back through smaller borders
        pi[i] = j;
    }
    return pi;
}

// all START positions (0-indexed) of pattern p inside text t
vector<int> matches(const string& t, const string& p) {
    int m = sz(p);
    vector<int> pi = prefixFunction(p + '#' + t), res;
    for (int i=m+1; i<sz(pi); i++)
        if (pi[i] == m) res.pb(i - 2*m); // i is in "combined" coords -> shift back
    return res;
}
```

manacher.cpp**MANACHER — ALL palindromic substrings in $O(n)$**

$d1[i]$ = # of ODD palindromes centered at i (= radius incl. center)

longest odd palindrome centered at i has length $2*d1[i] - 1$.

$d2[i]$ = # of EVEN palindromes centered BETWEEN $i-1$ and i

longest even one there has length $2*d2[i]$.

e.g. "abaa": $d1 = 1\ 2\ 1\ 1$, $d2 = 0\ 0\ 0\ 1$ ("aa" centered between index 2,3)

RECALL — same z-box idea as z-function: keep the rightmost known palindrome

$[l, r]$; inside it, position i mirrors to $l+r-i$, so $d[i]$ starts from the

mirror's value (capped by the box), then extends by brute force. $O(n)$ amortized.

INSTANT ANSWERS AFTER $O(n)$ BUILD:

- longest palindromic substring: max over $2*d1[i]-1$ and $2*d2[i]$.

- is $s[l..r]$ a palindrome, $O(1)$: ($len = r-l+1$)

len odd : $d1[(l+r)/2] \geq (len+1)/2$

len even: $d2[(l+r+1)/2] \geq len/2$

- total # of palindromic substrings = $\text{sum}(d1) + \text{sum}(d2)$.

```
vector<int> d1, d2;
```

```
void manacher(const string& s) {
    int n = sz(s);
    d1.assign(n, 0); d2.assign(n, 0);

    for (int i=0, l=0, r=-1; i<n; i++) {
        // odd lengths
        int k = (i > r) ? 1 : min(d1[l+r-i], r-i+1); // mirror value, capped
        while (0 <= i-k && i+k < n && s[i-k] == s[i+k]) k++; // extend
        d1[i] = k--;
        if (i+k > r) l = i-k, r = i+k; // new rightmost palindrome
    }

    for (int i=0, l=0, r=-1; i<n; i++) {
        // even lengths
        int k = (i > r) ? 0 : min(d2[l+r-i+1], r-i+1);
        while (0 <= i-k-1 && i+k < n && s[i-k-1] == s[i+k]) k++;
        d2[i] = k--;
        if (i+k > r) l = i-k-1, r = i+k;
    }
}
```

multiset-hashing.cpp**MULTISET HASHING — order-INDEPENDENT hashing (compare multisets in O(1))**

Question it answers: "do these two collections contain exactly the same elements with the same multiplicities?" — regardless of order.

(e.g. is substring A an ANAGRAM of substring B; are two subtrees equal as bags)

IDEA: give every DISTINCT VALUE a random 5-dimensional vector (Hash()).

The hash of a multiset = plain SUM of the vectors of its elements.

add element v -> cur = cur + H[v]

remove element v -> cur = cur - H[v]

Sum is commutative -> order can't matter. Forging a collision needs to hit

5 independent random $\sim 1e9$ coordinates -> probability ~ 0 .

USAGE:

```
map<int, Hash> H; auto getH = [&](int v){ if (!H.count(v)) H[v] = Hash(); return H[v]; };
Hash cur(0,0,0,0,0); // empty multiset (explicit zeros – Hash() is RANDOM!)
cur = cur + getH(v); // v enters
cur = cur - getH(v); // v leaves
sliding window / two windows -> compare with ==
```

NOTES:

- No mod anywhere: coords stay $< \sim 2e14$ for $2e5$ elements, fits ll fine.
- PITFALL: default constructor Hash() generates a NEW RANDOM vector – that's the per-value generator. The "zero" hash must be built as Hash(0,0,0,0,0).
- Works inside prefix sums too: $pre[i] = pre[i-1] + H[a[i]]$ -> hash of any subarray = $pre[r] - pre[l-1]$ -> "are these two ranges permutations of each other" in O(1).

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
template <typename T> T rand(T l, T r) { return uniform_int_distribution<T>(l, r)(rng); }
```

```
struct Hash {
    ll a, b, c, d, e;
    Hash(ll _a, ll _b, ll _c, ll _d, ll _e):a(_a),b(_b),c(_c),d(_d),e(_e){}
    Hash() { // RANDOM vector – use once per distinct value
        a = rand(1, (int)1e9);
        b = rand(1, (int)1e9);
        c = rand(1, (int)1e9);
        d = rand(1, (int)1e9);
        e = rand(1, (int)1e9);
    }
    Hash operator+(const Hash& other) const {
        return Hash(a+other.a,b+other.b,c+other.c,d+other.d,e+other.e);
    }
    Hash operator-(const Hash& other) const {
        return Hash(a-other.a,b-other.b,c-other.c,d-other.d,e-other.e);
    }
    bool operator==(const Hash& other) const {
        return a==other.a&&b==other.b&&c==other.c&&d==other.d&&e==other.e;
    }
};
```

trie.cpp

TRIE (prefix tree) — insert / search / delete words, $O(|\text{word}|)$ each

Node bookkeeping (this is what you usually query):

```
count = how many inserted words PASS THROUGH this node
    -> "how many words start with prefix p": walk p, return count (0 if walk dies)
isEnd = how many inserted words END exactly here (int, so duplicates are counted)
```

CHANGE HERE: N (alphabet size) + baseChar — 'a' for lowercase, 'A', '0' for digits...

PITFALLS:

- del(s) assumes s WAS inserted (call search first if unsure) — deleting an absent word corrupts counts. It removes ONE occurrence and frees dead branches.
- Memory: $\sim(\text{total inserted chars}) \text{ nodes} * (26 \text{ pointers} + 2 \text{ ints}) \approx 220\text{B}/\text{char}$.
1e6 total chars is fine; 1e7 is not — use arrays-of-int trie instead.
- Multi-test: no clear() here — recreate the Trie object (leaks, but CP doesn't care) or add a recursive delete.

COMMON EXTENSIONS (one-liners to add):

- countPrefix(p): walk p, return cur->count
- countWord(s): walk s, return cur->isEnd
- XOR problems on numbers -> binary-trie.cpp

```
class Trie {
    const static int N = 26;           // CHANGE: alphabet size
    const static int baseChar = 'a';   // CHANGE: first char of alphabet
    class Node {
    public:
        Node* child[N];
        int count, isEnd;
        Node():isEnd(false), count(0) {
            memset(child,0,sizeof child);
        }
    };
    Node* root = new Node();

public:
    void insert(const string& s) {
        Node* cur = root;
        for (char ch : s) {
            int c = ch - baseChar;
            if (cur->child[c] == NULL) cur->child[c] = new Node();
            cur = cur->child[c];
            cur->count++;           // one more word passes through
        }
        cur->isEnd++;
    }

    // internal: unwind the path, decrement counts, free branches that died
    void del(Node* cur, const string& s, int i) {

        if (i == (int)s.size()) {
            cur->isEnd--;
            return;
        }

        int c = s[i] - baseChar;
        Node* nxt = cur->child[c];
        del(nxt, s, i+1);

        nxt->count--;

        if (!nxt->count) {          // no word passes here anymore -> prune
            delete nxt;
            cur->child[c] = NULL;
        }
    }

    void del(const string& s) {      // !! only call if s is currently in the trie
        del(root,s,0);
    }

    bool search(const string& s) {   // whole-word lookup (prefix: return cur->count instead)
        Node* cur = root;
        for (char ch : s) {
            int c = ch - baseChar;
            if (cur->child[c] == NULL) return false;
            cur = cur->child[c];
        }
        return cur->isEnd;
    }
};
```

Geometry

closest-pair.cpp

CLOSEST PAIR OF POINTS — $O(n \log n)$ sweep line. Returns SQUARED distance.

Needs P / dist2 from geometry-basics.cpp. $n \geq 2$. Duplicates $\rightarrow$ returns 0.

RECALL — the sweep: process points left to right, keep an "active strip" of points whose x is within $D = \sqrt{\text{best}}$ of the current point, ordered by y (the set). For each new point only candidates with $|dy| \leq D$ matter — and a $D \times 2D$ box can only hold $O(1)$ points that are pairwise $\geq D$ apart, so the inner loop is amortized constant. best only shrinks $\rightarrow$ total $O(n \log n)$.

The only double-ish moment is computing $D = \text{ceil}(\sqrt{\text{best}})$ to bound the strip; all COMPARISONS stay in integers (dist2), so precision can't hurt correctness — D is just a (safe, rounded-up) search radius.

```
ll closestPair(vector<P> p) {
    sort(all(p));                                // by x (then y) — P's operator<
    set<pair<ll,ll>> strip;                        // {y, x} of points in the active strip
    ll best = LLONG_MAX;
    int j = 0;                                    // strip's left edge (index into p)
    for (int i=0; i<sz(p); i++) {
        ll D = (ll)ceil(sqrtl((long double)best));
        while (j < i && p[i].x - p[j].x >= D) // too far left to ever matter  $\rightarrow$  evict
            strip.erase({p[j].y, p[j].x}), j++;

        for (auto it = strip.lower_bound({p[i].y - D, LLONG_MIN});
             it != strip.end() && it->first <= p[i].y + D; ++it)
            best = min(best, dist2({it->second, it->first}, p[i]));

        strip.insert({p[i].y, p[i].x});
    }
    return best;                                // squared! sqrt only for output
}
```

geometry-basics.cpp

GEOMETRY BASICS — integer-only building blocks (no doubles until you must print)

Coordinates up to 1e9 are safe: cross/dot/dist2 peak at ~8e18, fits ll.

THE ONE TOOL THAT SOLVES 90% OF GEOMETRY — the cross product:

```
cross(a, b) = a.x*b.y - a.y*b.x      (of vectors)
cross(o, a, b) = cross(a-o, b-o), its SIGN tells you where b is
                  relative to the line o->a:
    > 0 -> b is to the LEFT (counter-clockwise turn o->a->b)
    < 0 -> b is to the RIGHT (clockwise turn)
    = 0 -> o, a, b COLLINEAR
|cross(o,a,b)| = 2 * area of triangle o,a,b.
```

dot(a, b) > 0: angle < 90; = 0: perpendicular; < 0: angle > 90.

PITFALLS:

- NEVER compare double distances – compare SQUARED ll distances (dist2).
- Read ints into ll immediately; subtraction before multiplication is what keeps everything in range.
- Polygon functions expect vertices in order (CW or CCW), no repeated last point.

```
struct P {
    ll x, y;
    P operator-(const P& o) const { return {x - o.x, y - o.y}; }
    P operator+(const P& o) const { return {x + o.x, y + o.y}; }
    bool operator<(const P& o) const { return x < o.x || (x == o.x && y < o.y); }
    bool operator==(const P& o) const { return x == o.x && y == o.y; }
};

ll cross(P a, P b) { return a.x*b.y - a.y*b.x; }
ll cross(P o, P a, P b) { return cross(a-o, b-o); }      // sign: see header
ll dot(P a, P b) { return a.x*b.x + a.y*b.y; }
ll dist2(P a, P b) { ll dx=a.x-b.x, dy=a.y-b.y; return dx*dx + dy*dy; } // SQUARED distance
int sgn(ll v) { return (v > 0) - (v < 0); }

// is p on segment a-b (endpoints included)? collinear + inside the bounding box
bool onSeg(P a, P b, P p) {
    return cross(a, b, p) == 0
        && min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x)
        && min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}

// do segments a-b and c-d share ANY point (crossing, touching, or overlapping)?
bool segIntersect(P a, P b, P c, P d) {
    int d1 = sgn(cross(c, d, a)), d2 = sgn(cross(c, d, b));
    int d3 = sgn(cross(a, b, c)), d4 = sgn(cross(a, b, d));
    if (d1 != d2 && d3 != d4) return true; // proper crossing: each seg separates the other's ends
    return onSeg(a, b, c) || onSeg(a, b, d) || onSeg(c, d, a) || onSeg(c, d, b); // touch / overlap cases
}

// TWICE the signed area of any simple polygon (shoelace).
// result > 0 -> vertices are CCW, < 0 -> CW. Real area = abs(...) / 2.0
// (keep the x2 version to stay in integers – area itself can be x.5)
ll area2(vector<P>& poly) {
    ll s = 0;
    for (int i=0, n=sz(poly); i<n; i++)
        s += cross(poly[i], poly[(i+1)%n]);
    return s;
}

// point vs simple polygon (ANY simple polygon, convex or not), O(n):
// returns 0 = outside, 1 = on the boundary, 2 = strictly inside (ray casting)
int inPolygon(vector<P>& poly, P p) {
    int n = sz(poly), cnt = 0;
    for (int i=0; i<n; i++) {
        P a = poly[i], b = poly[(i+1)%n];
        if (onSeg(a, b, p)) return 1;
        cnt ^= ((p.y < a.y) - (p.y < b.y)) * cross(p, a, b) > 0; // edge crosses the upward ray?
    }
    return cnt ? 2 : 0;
}
```

Other

bit-tricks.cpp**BIT TRICKS — counting set bits in ranges + XOR prefix, O(1) each**

(the one-line bit identities live in NOTES.md; this file is the actual code)

countSetBits(l, r, bit): how many numbers in [l, r] have `bit` set.

RECALL — bit b cycles with period $2^{(b+1)}$: first half off, second half on.

Count in [0, x] = full periods * half + the partial period's overlap; subtract prefixes.

Total set bits in a range: loop this over all 60 bits (still $\sim O(60)$).

Typical use: sum/xor contributions bit by bit ("count pairs whose AND has bit b", ...).

xorUpTo(n) = $1 \oplus 2 \oplus \dots \oplus n$. RECALL — it's periodic with period 4:

$n \% 4 == 0 \rightarrow n, \quad == 1 \rightarrow 1, \quad == 2 \rightarrow n+1, \quad == 3 \rightarrow 0$

XOR of any range [l, r] = xorUpTo(r) $\oplus$ xorUpTo(l-1) (prefix trick, xor is its own inverse).

```

ll onesUpTo(ll x, int bit) {           // # of v in [0, x] with `bit` set
    if (x < 0) return 0;
    ll len = 1LL << (bit + 1), half = 1LL << bit;
    return (x / len) * half + max(0LL, x % len - half + 1);
}

ll countSetBits(ll l, ll r, int bit) { // # of v in [l, r] with `bit` set
    return onesUpTo(r, bit) - onesUpTo(l - 1, bit);
}

ll xorUpTo(ll n) {                     //  $1 \oplus 2 \oplus \dots \oplus n$  (n >= 0)
    switch (n % 4) {
        case 0: return n;
        case 1: return 1;
        case 2: return n + 1;
        default: return 0;
    }
}

```

coordinate-compression.cpp**COORDINATE COMPRESSION — map big values to 0..k-1, order preserved. $O(n \log n)$** WHEN: values up to $1e9/1e18$ but only n of them, and you need them as ARRAYINDICES (Fenwick, segtree, cnt[] in Mo's, ...). Comparisons still work because the mapping is monotonic: $a[i] < a[j]$ before $\Leftrightarrow$ after.

SNIPPET (inside solve). After it:

```

a[i] = compressed rank in [0, k)   where k = sz(d)
d[a[i]] = the ORIGINAL value (keep d around to answer/output real values!)
rank of arbitrary x: lower_bound(all(d), x) - d.begin()

```

NOTE: compressing MULTIPLE arrays together (queries + array values)?

Concatenate everything into d before sort+unique, then rank each against d.

```

vector<int> d = a;
sort(d.begin(), d.end());
d.resize(unique(d.begin(), d.end()) - d.begin()); // sorted distinct values
for (int i = 0; i < n; ++i) {
    a[i] = lower_bound(d.begin(), d.end(), a[i]) - d.begin(); // value -> its rank
}

```

ordered-multiset.cpp**ORDERED SET / MULTISSET (GNU pb_ds) — std::set + ORDER STATISTICS in $O(\log n)$**

The two extra superpowers over std::set:

s.order_of_key(x) -> how many elements are STRICTLY LESS than x (x's rank)
 s.find_by_order(i) -> iterator to the i-th smallest (0-indexed); *it to read
 -> "k-th smallest" and "count elements < x" without a Fenwick.

ordered_set<T> : unique elements, everything works like std::set.

ordered_multiset<T> : duplicates via the less_equal comparator HACK — the tree thinks equal elements are "different", which BREAKS some members:

!! find(x) -> ALWAYS returns end() (never use)
 !! erase(x) -> silently erases NOTHING (never use)
 !! lower_bound / upper_bound SWAP meanings (lower_bound acts like upper_bound & vice versa)
 OK: insert, order_of_key (still = count of elements < x), find_by_order.

ERASE ONE COPY of x (the only safe way):

```
auto it = ms.find_by_order(ms.order_of_key(x)); // iterator to first occurrence of x
if (it != ms.end() && *it == x) ms.erase(it); // guard: x might be absent
```

Include the two headers below (they're not in bits/stdc++.h on some setups).

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
```

```
template<class T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
```

```
template<class T>
using ordered_multiset = tree<T, null_type, less_equal<T>, rb_tree_tag,
tree_order_statistics_node_update>;
```

random.cpp

RANDOM — the ONLY rng you should use in CP (never rand()/srand: weak + 32767 cap on MSVC)

rand(l, r) -> uniform integer in [l, r], BOTH INCLUSIVE. Type follows the args:

rand(1, 100) -> int rand(1LL, (1L)1e18) -> ll (don't mix int and ll args!)

OTHER USES:

```
shuffle(all(v), rng);           // kill adversarial input order (anti-quicksort tests)
rng() by itself                 // raw random 64-bit number
```

WHEN randomness saves you:

- random bases/keys for hashing (Strings/ hashing, multiset-hashing)
- randomized algorithms: pick random pivot/sample, "guess a likely answer" tricks
- stress-testing: generate small random tests to diff brute force vs solution

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
template <typename T> T rand(T l, T r) { return uniform_int_distribution<T>(l, r)(rng); }
```